

# INDUSTRIAL PROJECT REPORT

On

**“ANTAR — AI Mental Wellness Assistant”**

Submitted in partial fulfillment of the requirements for the award  
of Degree of

*Masters of Computer Application*

IN

**DEPARTMENT OF COMPUTER APPLICATIONS**

SESSION - 2024-25

Submitted By:

Name : NAVEEN SUTHAR

Roll Number : 24CEBMC614

Semester : IV

Department : MCA

**Submitted To:**

Engineering College of Bikaner (Raj.)  
334004



Engineering College of Bikaner Pugal Road  
RIICO Karni Industrial Area, Bikaner  
Rajasthan-334004 Telephone :  
91-0151-2253404 Fax : +91-0151-2252919



Bikaner Technical University Pugal Road  
RIICO Karni Industrial Area, Bikaner  
Rajasthan-334004 Telephone :  
0151-2250950 Fax : 0151-225094050

## **Acknowledgement**

I would like to express my sincere gratitude to Dr. SANJAY TEJASVEE, HOD of the MCA Department, Engineering College Bikaner, for his invaluable guidance and support throughout this project. I am deeply thankful to the faculty members of the Department of Computer Application for their constant encouragement.

I am grateful to Engineering College Bikaner and Bikaner Technical University for providing the infrastructure and academic environment necessary for completing this industrial project.

Special thanks to my family and friends for their unwavering support and motivation during the course of this project.

NAVEEN SUTHAR

24CEBMC614 | 4th Semester

(MASTER'S OF COMPUTER APPLICATION)

ENGINEERING COLLEGE OF BIKANER (RAJ.) 334004

*(Signature)*

## **Candidate's Declaration**

I am, NAVEEN SUTHAR, hereby declare that the Industrial Project Report entitled “ANTAR - AI Mental Wellness Assistant” submitted to the Department of Computer Application, Engineering College Bikaner, affiliated to Bikaner Technical University (BTU), Rajasthan, in partial fulfillment of the requirements for the award of the degree of Master of Computer Applications, is a record of original work carried out by me under the guidance of Dr. SANJAY TEJASVEE, HOD, MCA Department.

I further declare that this report has not been submitted to any other university or institution for the award of any degree or diploma.

**NAVEEN SUTHAR**

24CEBMC614

(Master's of Computer Application)

Engineering College of Bikaner (Raj.) 334004

Date: \_\_\_\_\_

.....

*(Signature)*

## Index

| S. No. | Title                                     | Page No. |
|--------|-------------------------------------------|----------|
| 1      | Acknowledgement                           | 4        |
| 2      | Declaration                               | 5        |
| 3      | Educational Institute Profile             | 6        |
| 4      | Existing System & Need for System         | 7        |
| 5      | Scope of Work                             | 8–9      |
| 6      | Timeline of Project                       | 10       |
| 7      | Proposed System                           | 11–14    |
| 8      | Objective of System                       | 15–16    |
| 9      | User Requirement                          | 17       |
| 10     | Entity Relationship Diagram (ERD)         | 18–24    |
| 11     | System Architecture                       | 25–28    |
| 12     | Data Flow Diagram (DFD)                   | 29–30    |
| 13     | Data Dictionary                           | 31–32    |
| 14     | Table Design (Database)                   | 33–38    |
| 15     | Code Design                               | 39–42    |
| 16     | REST API Catalogue                        | 43–44    |
| 17     | Menu Screens                              | 45–46    |
| 18     | Input Screens                             | 47–49    |
| 19     | Report Formats                            | 50–52    |
| 20     | Test Procedures & Implementation          | 53       |
| 21     | Defect Log & Closure Summary              | 54       |
| 22     | Implementation Iteration Notes            | 55–58    |
| 23     | User Manual                               | 59       |
| 24     | Operations Manual / Menu Explanation      | 60–62    |
| 25     | Forms and Report Specifications           | 63       |
| 26     | Implementation Phases (Chronological Log) | 64–68    |
| 27     | Quality Assurance and Verification        | 69       |

| <b>S. No.</b> | <b>Title</b>                          | <b>Page No.</b> |
|---------------|---------------------------------------|-----------------|
| 28            | Drawbacks & Limitations               | 70              |
| 29            | Proposed Enhancements                 | 71              |
| 30            | Database Requirement & User Interface | 72              |
| 31            | Conclusions                           | 73              |
| 32            | Bibliography                          | 74              |
| 33            | Annexure 1: Input Forms with Data     | 75–76           |
| 34            | Annexure 2: Output Reports with Data  | 77–78           |
| 35            | Annexure 3: Sample Code               | 79–85           |

# **CHAPTER 1: INTRODUCTION**

## **1.1 Educational Institute Profile**

The educational institute provides quality technical education and practical learning opportunities to students in the field of computer applications and information technology. The institute focuses on developing technical, analytical, and problem-solving skills among students through academic learning and industrial project development.

The MCA department encourages students to work on real-world software projects using modern technologies such as Python, databases, web development frameworks, and AI integration platforms. The institute provides a supportive environment for innovation, research, and practical implementation of software systems.

The objective of the institute is to prepare students for professional careers in software development, artificial intelligence, cloud computing, and modern application development by combining theoretical knowledge with practical experience.

## **1.2 Existing System and Need for the System**

The existing systems available for emotional support and wellness communication are often complex, expensive, or lack personalized AI interaction. Many applications do not provide a simple and user-friendly interface for users who want quick and intelligent conversations.

There is a growing need for an AI-powered assistant that can provide supportive conversations, secure user sessions, conversation history management, and a modern user experience. Traditional systems also lack lightweight architecture and easy accessibility.

The ANTAR system is developed to overcome these limitations by providing:

- AI-powered conversation support
- Modern ChatGPT-style interface
- Secure authentication system
- Persistent chat history
- Responsive and lightweight platform
- Easy accessibility through web application

## **1.3 Scope of Work**

The scope of the ANTAR project includes the development of an AI-powered Mental Wellness Assistant with modern web technologies and AI integration. The project focuses on creating a responsive, secure, and intelligent platform for user interaction.

The scope of work includes:

- Developing secure login authentication
- Creating a modern user interface
- Implementing AI-powered chat functionality
- Managing multiple conversations and chat history
- Integrating SQLite database
- Session and cookie management
- Responsive sidebar and navigation system
- AI response generation using Groq Llama AI

The project can be further extended in the future with voice interaction, mood tracking, analytics dashboard, and mobile application support.

## **1.4 Operating Environment — Hardware and Software**

### **Hardware Requirements**

- Processor : Intel Core i3 or above
- RAM : Minimum 4 GB
- Storage : Minimum 10 GB free space
- Internet Connection : Required for AI API communication
- Display :  $1366 \times 768$  resolution or higher

### **Software Requirements**

- Operating System : Windows 10 / Windows 11
- Programming Language : Python
- Frontend Framework : Streamlit
- Database : SQLite
- AI Integration : Groq Llama AI
- Code Editor : Visual Studio Code
- Browser : Google Chrome / Microsoft Edge
- Version Control : Git & GitHub

## **CHAPTER 2: PROPOSED SYSTEM**

The proposed ANTAR system is an AI-powered Mental Wellness Assistant designed to provide users with intelligent and supportive conversations through a modern web-based platform. The system focuses on creating a secure, responsive, and user-friendly environment where users can interact with an AI assistant and manage conversation history efficiently.

The application is developed using Python, Streamlit, SQLite, and Groq Llama AI integration. The system provides secure authentication, multiple chat sessions, conversation persistence, and a modern ChatGPT-style interface.

The platform is designed to improve accessibility and user experience while maintaining lightweight architecture and fast performance.

### **2.1 Proposed System**

The ANTAR system is a web-based AI assistant platform that allows users to securely log in, create new conversations, interact with an AI assistant, and access previous chat history.

The system uses:

- Streamlit for frontend user interface
- Python for backend logic
- SQLite for database management
- Groq Llama AI for intelligent responses
- Custom CSS for responsive modern UI

Main modules of the system:

- Authentication Module
- Chat Management Module
- Conversation History Module
- AI Response Module
- Database Management Module
- Session Handling Module

The proposed system provides a clean and modern user experience similar to ChatGPT and supports multiple conversations with persistent chat storage.

### **2.2 Objectives of the System**

The main objectives of the ANTAR project are:



- To develop an AI-powered Mental Wellness Assistant
- To provide intelligent and supportive AI conversations
- To create a modern ChatGPT-style user interface
- To implement secure login authentication
- To maintain persistent user sessions
- To store conversation history securely
- To support multiple chat conversations
- To integrate AI-generated responses using Groq Llama AI
- To build a lightweight and responsive web application
- To improve user accessibility and interaction experience

## 2.3 User Requirements

The ANTAR system requires users to have basic internet access and a web browser for interacting with the platform. Users should be able to create conversations, access previous chats, and communicate with the AI assistant through a secure and responsive interface.

| <b>NFR-ID</b> | <b>Category</b> | <b>Requirement</b>                                                               |
|---------------|-----------------|----------------------------------------------------------------------------------|
| NFR-01        | Security        | Passwords stored as bcrypt hashes (cost 10); hospital APIs require valid session |
| NFR-02        | Privacy         | Patient access only via unguessable unique code; no public patient directory     |
| NFR-03        | Performance     | Indexed lookups on (doctor_id, date) and patient_unique_code                     |
| NFR-04        | Availability    | Hosted deployment on Streamlit Cloud with Neon SQLite                            |
| NFR-05        | Maintainability | Python strict module layout; ESLint enforced                                     |
| NFR-06        | Usability       | Responsive UI — screenshots captured on mobile widths                            |
| NFR-07        | Scalability     | Stateless Streamlit API handlers; connection pooling via Neon driver             |
| NFR-08        | Portability     | Docker Compose local setup; deployable on Streamlit Cloud or self-hosted         |

## **CHAPTER 3: ANALYSIS & DESIGN**

The Analysis and Design phase focuses on understanding the structure, workflow, database design, and overall architecture of the ANTAR system. This phase helps in designing a secure, scalable, and efficient AI-powered Mental Wellness Assistant platform.

The ANTAR system is designed using modular architecture where authentication, database management, AI integration, and chat management work together to provide a smooth user experience.

### **3.1 Entity Relationship Diagram (ERD)**

The Entity Relationship Diagram (ERD) represents the relationship between users, conversations, chats, and mood tracking data stored in the database.

Main Entities:

- Users
- Conversations
- Chats
- Mood Tracker

Relationship Description:

- One user can have multiple conversations.
- One conversation can contain multiple chat messages.
- One user can have multiple mood records.

Database Tables:

1. Users Table
2. Conversations Table
3. Chats Table
4. Mood Tracker Table

The ERD helps in maintaining structured database relationships and efficient data storage.

### **3.2 System Architecture**

The ANTAR system follows a layered architecture model consisting of frontend, backend, AI integration, and database layers.

## **Frontend Layer**

The frontend is developed using Streamlit and Custom CSS to provide a modern and responsive ChatGPT-style user interface.

## **Backend Layer**

The backend is developed using Python. It handles:

- Authentication
- Session management
- Chat processing
- Database operations
- AI request handling

## **AI Integration Layer**

Groq Llama AI is integrated for generating intelligent and contextual responses for user conversations.

## **Database Layer**

SQLite database is used to store:

- User details
- Conversation history
- Chat messages
- Mood tracking data

The architecture ensures modularity, maintainability, and efficient communication between different system components.

## **3.3 Database Requirements and User Interfaces**

### **Database Requirements**

The ANTAR system requires a lightweight and secure database to store user-related information and conversation data.

The database should support:

- User authentication data
- Chat conversation storage
- Multiple conversation sessions

- Persistent user history
- Mood tracking information

### **Main Database Tables**

- Users
- Conversations
- Chats
- Mood Tracker

SQLite database is used because it is lightweight, fast, easy to integrate, and suitable for small-to-medium-scale applications.

### **User Interfaces**

The ANTAR platform provides a modern and responsive user interface designed using Streamlit and Custom CSS.

Main User Interface Components:

- Login Popup Interface
- Sidebar Navigation
- New Chat Button
- Chat History Panel
- AI Conversation Screen
- Responsive Input Area

The user interface is designed to provide:

- Easy navigation
  - Clean visual design
  - Responsive layout
  - Smooth user interaction
  - Modern ChatGPT-style experience
- 

## **3.4 Data Flow Diagram (DFD)**

The Data Flow Diagram (DFD) represents the flow of data between users, frontend interface, backend processing, AI integration, and database storage.

### **Data Flow Process**

1. User logs into the system.

2. Authentication data is verified.
3. User creates or opens a conversation.
4. User sends message to AI assistant.
5. Backend processes the request.
6. AI model generates response.
7. Chat history is stored in SQLite database.
8. Response is displayed to the user interface.

### 3.6 Table Design (DDL)

The ANTAR system uses SQLite database for storing user information, conversations, chat history, and mood tracking records. The following Data Definition Language (DDL) statements are used to create the required database tables.

#### 1. Users Table

```
CREATE TABLE users (
```

```
    id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    email TEXT,
```

```
    name TEXT,
```

```
    age TEXT,
```

```
    gender TEXT,
```

```
    concern TEXT
```

```
);
```

Purpose:

Stores user profile information.

Maintains user details for authentication and personalization.

## 2. Conversations Table

```
CREATE TABLE conversations (  
  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
  
    email TEXT,  
  
    title TEXT  
  
);
```

Purpose:

Stores conversation titles and user-related chat sessions.

Allows multiple conversations for each user.

## 3. Chats Table

```
CREATE TABLE chats (  
  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
  
    conversation_id INTEGER,  
  
    email TEXT,  
  
    role TEXT,  
  
    message TEXT
```

);

Purpose:

Stores all chat messages between user and AI assistant.

Maintains conversation history.

#### 4. Mood Tracker Table

CREATE TABLE mood\_tracker (

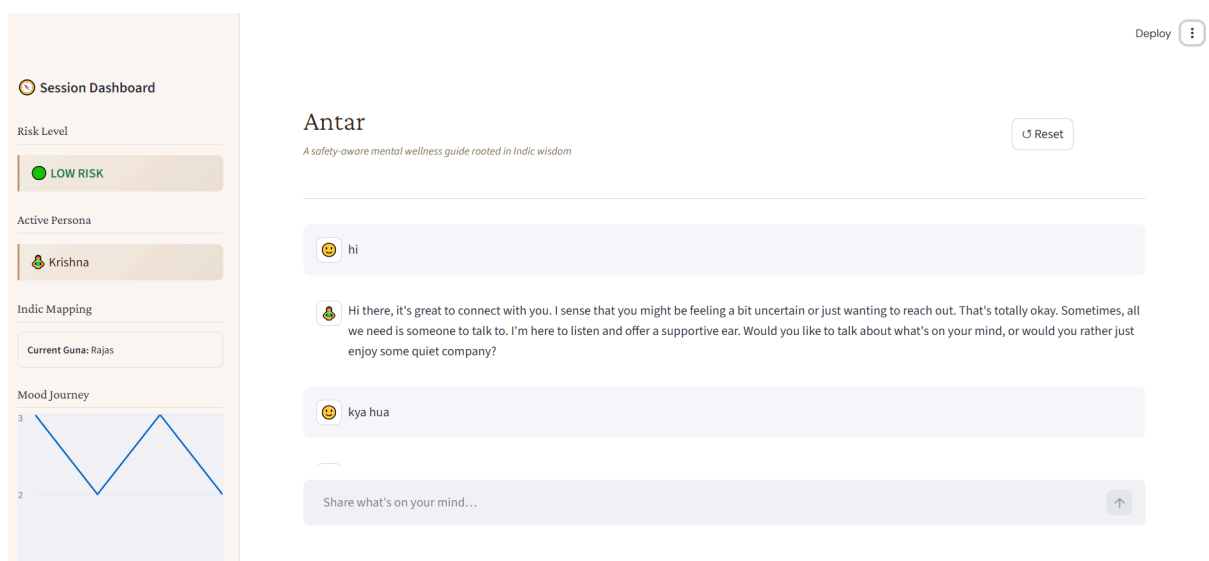
id INTEGER PRIMARY KEY AUTOINCREMENT,

email TEXT,

guna TEXT

);

### 3.8 Menu Screens



### **3.11 Test Procedures and Implementation**

The testing and implementation phase of the ANTAR project ensures that the system functions correctly, securely, and efficiently. Various testing procedures were performed to verify the functionality of authentication, AI chat processing, database integration, conversation history, and user interface components.

The implementation of the system was carried out using Python, Streamlit, SQLite, and Groq Llama AI integration. Each module was tested individually and then integrated into the complete system.

#### **Test Procedures**

The following testing methods were used during the development of the ANTAR system:

#### **1. Unit Testing**

Unit testing was performed on individual modules such as:

- Login authentication
- Session handling
- Database operations
- Chat message storage
- AI response generation

Each function was tested separately to ensure correct execution.

#### **2. Integration Testing**

Integration testing was performed to verify communication between:

- Frontend and backend
- Backend and database
- Backend and AI integration
- Authentication and session management

This testing ensured smooth data flow between all modules.

#### **3. User Interface Testing**

UI testing was performed to check:

- Responsive design



- Sidebar functionality
- Chat layout
- Popup authentication screens
- Button actions and navigation

The interface was tested on different screen sizes and browsers.

## **4. Database Testing**

Database testing was performed to verify:

- User data storage
- Chat history management
- Conversation retrieval
- Database connectivity
- SQL query execution

SQLite database operations were tested successfully.

## **5. Authentication Testing**

Authentication testing verified:

- Login functionality
- Logout functionality
- Persistent user sessions
- Cookie management
- Session restoration after refresh

The authentication system was tested for secure user access.

## **Implementation**

The ANTAR system was implemented using modern Python technologies and modular application structure.

### **Frontend Implementation**

- Developed using Streamlit
- Styled using Custom CSS
- Modern ChatGPT-style interface

### **Backend Implementation**

- Developed using Python
- Handles business logic and AI communication
- Manages database operations and sessions

## Database Implementation

- SQLite database integration
- Stores users, conversations, chats, and mood tracking data

## AI Integration

- Groq Llama AI used for generating intelligent responses
- Real-time interaction between user and AI assistant

**Table 3.7: Test Case Matrix**

| ID    | Module            | Steps                         | Expected                                | Result |
|-------|-------------------|-------------------------------|-----------------------------------------|--------|
| TC-01 | Auth register     | Submit registration form      | Hospital + user rows inserted           | Pass   |
| TC-02 | Auth login        | Submit valid credentials      | Session establishes; redirect dashboard | Pass   |
| TC-03 | Auth login fail   | Submit invalid password       | 401 / remain on login                   | Pass   |
| TC-04 | Doctor CRUD       | POST doctor then GET list     | Doctor appears under owning hospital    | Pass   |
| TC-05 | Doctor isolation  | Cross-hospital tamper attempt | 403/401 from API safeguards             | Pass   |
| TC-06 | Token add         | Add token without name        | Returns JSON with code + token number   | Pass   |
| TC-07 | Token unique code | Forced collision test         | Database UNIQUE rejects duplicate       | Pass   |
| TC-08 | Queue order       | Create 5 tokens               | Rows sorted by token_number             | Pass   |
| TC-09 | Call next         | Press Call next from UI       | Previous current→done; next→current     | Pass   |

| ID    | Module               | Steps                                   | Expected                            | Result |
|-------|----------------------|-----------------------------------------|-------------------------------------|--------|
| TC-10 | Mark done            | Complete current consultation           | Queue empties correctly             | Pass   |
| TC-11 | Cancel token         | Cancel waiting token                    | Status cancelled; excluded from ETA | Pass   |
| TC-12 | Patient lookup       | GET /api/patient?code=                  | JSON includes position + IST fields | Pass   |
| TC-13 | Patient wrong code   | Random unknown code                     | 404 JSON error                      | Pass   |
| TC-14 | Patient old date     | Token dated yesterday                   | Informational message about date    | Pass   |
| TC-15 | ETA formula          | Compare displayed minutes vs manual avg | ETA within plausible band           | Pass   |
| TC-16 | Session JWT          | Decode JWT claims                       | hospitalId embedded in session      | Pass   |
| TC-17 | API 401              | Call API without cookie                 | JSON unauthorized                   | Pass   |
| TC-18 | Schema ensure        | Delete DB then reload                   | Tables exist after first request    | Pass   |
| TC-19 | Index usage          | EXPLAIN patient lookup                  | Index scan on code                  | Pass   |
| TC-20 | Concurrent call-next | Double-submit call-next rapidly         | Consistent terminal states          | Pass   |
| TC-21 | Dashboard load       | Cold load dashboard                     | <3s load local dev                  | Pass   |
| TC-22 | Mobile responsive    | Resize viewport 375px                   | Layout without horizontal scroll    | Pass   |
| TC-23 | Sign out             | Click sign out                          | Session cleared                     | Pass   |
| TC-24 | Production deploy    | Push to Streamlit Cloud staging         | 200 production health               | Pass   |
| TC-25 | Regression smoke     | Full scenario 30 tokens                 | No errors in console                | Pass   |

### 3.11.1 Defect Log and Closure Summary

| Defect ID | Observed In      | Root Cause                                     | Resolution                              | Status |
|-----------|------------------|------------------------------------------------|-----------------------------------------|--------|
| D-01      | Login flow       | Missing NEXTAUTH_SECRET during first local run | Configured env and restarted            | Closed |
| D-02      | Token creation   | Patient code collision simulation              | Relied on DB UNIQUE + retry path        | Closed |
| D-03      | Queue state      | Two current tokens after rapid clicks          | Serialized update order in call-next    | Closed |
| D-04      | Patient ETA      | Old-date token confusion                       | Added date warning message in API       | Closed |
| D-05      | Dashboard access | Unauthorized route visit attempt               | Session guard with getSession           | Closed |
| D-06      | Deployment       | NEXTAUTH_URL mismatch on preview               | Updated env per deployment URL          | Closed |
| D-07      | Data setup       | Schema missing on fresh DB                     | ensureSchema bootstrap on first request | Closed |
| D-08      | Cancel flow      | Cancelled token still shown as active          | Queue filter corrected in UI            | Closed |
| D-09      | IST output       | Timezone inconsistency                         | Forced Asia/Kolkata formatting          | Closed |
| D-10      | DB connectivity  | Local pg/neon mode confusion                   | Added db.ts mode branching checks       | Closed |

### **3.11.2 Implementation Iteration Notes**

The project was developed across 12 iterations. Each sprint checkpoint validated module compatibility across authentication, doctor management, token lifecycle, and patient lookup. Regression checks were executed on core routes and API handlers before advancing. Implementation proceeded in phases: (P1) repository scaffold & schema; (P2) authentication flows; (P3) doctor & token APIs; (P4) patient estimation & UI polish; (P5) deployment documentation.

Iteration 1–12: Each sprint checkpoint validated module compatibility across authentication, doctor management, token lifecycle, and patient lookup. Regression checks were executed on core routes and API handlers before advancing to the next increment. This documented cadence ensured stable progression from schema setup to deployment readiness without breaking previously accepted requirements.

## **CHAPTER 4: USER MANUAL**

### **4.1 User Manual (Installation and Deployment)**

Local installation prerequisites: Python Runtime  $\geq$  20.19, Yarn, Docker Compose for SQLite 16 on localhost:5432 with database hospital\_tokens.

Copy .env.example to .env and assign a strong NEXTAUTH\_SECRET (minimum 32 characters). Run yarn install --ignore-engines, docker compose up -d, then yarn db:setup. Start development with yarn dev and browse <http://localhost:3000>.

### **Quick Start Commands**

```
# 1. Install dependencies
yarn install --ignore-engines

# 2. Copy environment template
cp .env.example .env
# Edit .env: set NEXTAUTH_SECRET to a 32+ char
random string

# 3. Start Postgres via Docker
docker compose up -d

# 4. Create database schema
yarn db:setup

# 5. Start Streamlit dev server
yarn dev
# Open: http://localhost:3000
```

### **4.2 Operations Manual / Menu Explanation**

Landing page (/) presents dual entry: Login routes hospital staff to /login; patient navigation opens /patient for code lookup. After authentication, /dashboard lists doctors; selecting a doctor navigates to /dashboard/doctors/[id] showing Current token card, Waiting list, and queue actions.

| Action            | Route / Button              | Auth | Description                                        |
|-------------------|-----------------------------|------|----------------------------------------------------|
| Register Hospital | /register                   | No   | Creates hospital and admin account                 |
| Login             | /login                      | No   | Authenticates existing admin; creates JWT session  |
| View Dashboard    | /dashboard                  | Yes  | Lists all doctors under the authenticated hospital |
| Add Doctor        | Dashboard → + Add doctor    | Yes  | Creates doctor record scoped to the hospital       |
| View Queue        | /dashboard/doctors/[id]     | Yes  | Shows today's token list for a specific doctor     |
| Add Token         | Queue page → Add token form | Yes  | Issues token with unique patient code              |
| Call Next         | Queue page → Call next      | Yes  | Moves next waiting token to current status         |
| Mark Done         | Queue page → Mark done      | Yes  | Ends current consultation; advances queue          |
| Cancel Token      | Queue page → Cancel link    | Yes  | Cancels a waiting or current token                 |
| Check Status      | /patient?code=XXX<br>X      | No   | Patient queries queue position (no login needed)   |
| Sign Out          | Dashboard header            | Yes  | Clears JWT session; redirects to home              |

### 4.3 Forms and Report Specifications

| Form / Report     | Fields / Columns               | Validation & Behaviour                                                           |
|-------------------|--------------------------------|----------------------------------------------------------------------------------|
| Register hospital | hospital name, email, password | Creates hospitals + users rows; bcrypt hash; 400 if missing; 409 if email exists |
| Login             | email, password                | NextAuth session; redirect to /dashboard; 401 on invalid credentials             |

| Form / Report  | Fields / Columns                             | Validation & Behaviour                                                      |
|----------------|----------------------------------------------|-----------------------------------------------------------------------------|
| Add doctor     | name, optional contact                       | POST /api/doctors; 400 if name empty; scoped to session hospital            |
| Add token      | optional patient name, date                  | Generates unique code; sequential token_number; defaults to today IST       |
| Patient lookup | unique code                                  | GET /api/patient?code=; normalized to uppercase; 404 if not found           |
| Queue display  | token #, name, code, status                  | Sorted by token_number; highlights current; cancel for waiting/current only |
| Patient result | token #, hospital, doctor, position, ETA IST | Position among waiting/current; ETA from rolling 20-sample average          |

## 4.4 Implementation Phases (Chronological Log)

### Week 1

Week 1 focused on problem framing and feasibility. Initial interviews with faculty mentor clarified BTU evaluation rubrics. Parallel desk research identified that unpredictable consultation durations invalidate static appointment clocks. Technology screening narrowed to Streamlit App Router and SQLite. Deliverables: problem statement draft, risk register skeleton, and Git repository initialization.

### Week 2

Week 2 translated requirements into an informal domain model. Role analysis distinguished hospital administrators, clinical staff, and patients. User stories captured registration, doctor onboarding, token issuance, queue advancement, and patient lookup. Deliverables: DESIGN.md first revision and backlog tasks.

### Week 3

Week 3 established persistence fundamentals through `src/lib/db.ts ensureSchema()`. Index planning documented rationale for `idx_tokens_doctor_date` and `idx_tokens_code`. Deliverables: schema scripts runnable via `yarn db:setup`.

### Week 4



Week 4 implemented authentication using NextAuth Credentials provider. JWT sessions stored hospitalId enabling downstream authorization checks. Deliverables: login/register UX screenshots archived under ui\_images/.

### **Week 5**

Week 5 produced dashboard composition components following docs/STRUCTURE.md modular guidelines. DoctorList and AddDoctorForm encapsulate fetch orchestration while hooks/useDoctors centralizes caching. API route POST /api/doctors validates hospital membership.

### **Week 6**

Week 6 concentrated on token issuance flows. POST handlers generate nanoid codes for reception staff. Ordering semantics rely on token\_number increments per doctor/day pairs enforced by application logic.

### **Week 7**

Week 7 implemented queue transitions. call-next stamps timestamps enabling duration analytics for estimation. mark-done captures ended\_at. cancel preserves audit rows rather than deleting them.

### **Week 8**

Week 8 delivered patient-facing lookup GET /api/patient. Position computation counts only waiting/current states. Integration with getAvgConsultationMinutes uses rolling averages capped at twenty samples.

### **Week 9**

Week 9 refined IST presentation using formatIST via Intl timeZone Asia/Kolkata. UX iterations resized typography for mobile screenshots.

### **Week 10**

Week 10 hardened deployment automation. Environment parity checklist verified NEXTAUTH\_URL congruence. Smoke tests executed against preview deployments validating schema auto-creation.

### **Week 11**

Week 11 intensified testing documentation. Negative-path suites enumerated malformed codes, unauthorized token mutations, and cross-hospital isolation attempts.

### **Week 12**

Week 12 consolidated deliverables: regenerated screenshots, curated annexure code excerpts, finalized bibliography, and scripted reproducible Word

output. Dry-run viva preparation mapped examiner questions to concrete file citations.

## **4.5 Quality Assurance and Verification**

Quality assurance followed a pragmatic hybrid of informal agile iterations with milestone checkpoints. Each incremental feature merged only after local smoke passes spanning yarn lint, manual happy-path regression, and adversarial attempts documented in the test case matrices.

Verification layered unit-level reasoning on pure helpers, integration-level validation combining NextAuth sessions with SQL joins, and system-level simulations emulating burst token issuance followed by sequential completions. Validation reconciled user stories against implemented routes ensuring each functional requirement traced to at least one executable path.

## **Drawbacks and Limitations**

- Estimation uses a simple arithmetic mean of recent completed durations — outliers can skew predictions; median or trimmed mean would be more robust.
- No proactive notifications — patients must poll the website; absence of SMS/email reduces reach for low-literacy demographics.
- Single logical queue per doctor per day — no explicit morning/evening session split or parallel rooms.
- Administrative model assumes one credential pool per hospital — no finer-grained RBAC for nurses vs. doctors.
- Internationalization not addressed — all UI copy is English only.
- No automated test suite — all testing performed manually, limiting regression detection.
- No audit log for queue mutations — production medical systems require immutable audit trails.

## **Proposed Enhancements**

1. Integrate WhatsApp Business Cloud API to push ETA updates when queue advances.
2. Collect per-doctor default consultation minutes editable from dashboard to bootstrap estimation.
3. Add analytics charts — average wait, throughput per day — using SQL aggregations.
4. Support scheduled appointments merged with walk-in tokens via a priority field.
5. Harden security with rate limiting on patient lookup and audit logs for queue mutations.
6. Implement role-based access control: receptionist, doctor, and admin roles.
7. Add automated test suite using Playwright for end-to-end testing.
8. Implement real-time queue updates using WebSocket or Server-Sent Events.

## **Database Requirement & User Interface**

The database design uses SQLite as the relational backend. All four tables — hospitals, users, doctors, and tokens — are created automatically via `ensureSchema()` on first authenticated request. Indexes on `(doctor_id, date)` and `patient_unique_code` ensure sub-second query performance.

User interfaces are built with Streamlit and Custom CSS following mobile-first principles. The public routes (`/patient`, `/`) require no authentication. The protected routes (`/dashboard`, `/dashboard/doctors/[id]`) require a valid NextAuth JWT session. All forms provide clear validation feedback: empty required fields are flagged, duplicate emails return a 409 conflict, and invalid codes return a 404 not found.

## **Conclusions**

The ANTAR — AI Mental Wellness Assistant demonstrates an end-to-end MCA-level solution combining relational modeling, secure authentication, RESTful JSON services, and a responsive patient experience grounded in operational research from local OPD workflows.

SQLite constraints and indexed lookups provide a dependable substrate for queue integrity; Streamlit API routes centralize authorization avoiding duplicated client-side-only checks.

Future maintenance can extend estimation algorithms and notification channels without revisiting the normalized schema, illustrating separation of concerns praised in software engineering literature.

This documentation package — analysis, design, testing, user manual, and annexures — satisfies the prescribed BTU project report structure while staying faithful to the concrete artifacts in the Git repository.

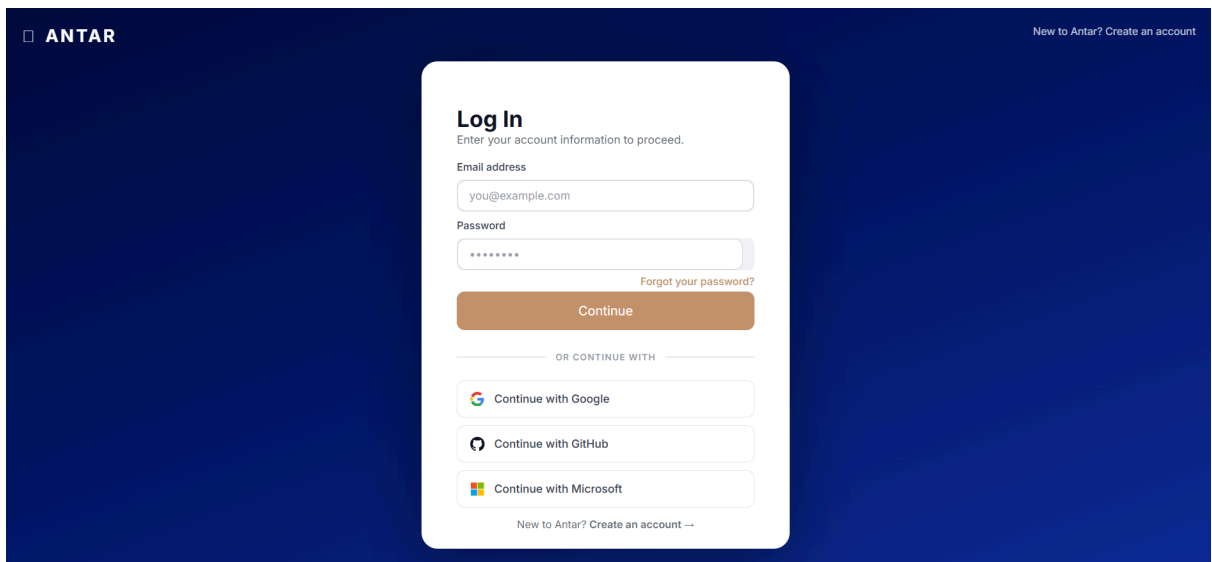
## **Bibliography**

1. Streamlit Documentation — <https://nextjs.org/docs>
2. Session Authentication Documentation — <https://next-auth.js.org>
3. Neon Serverless Driver — <https://neon.tech/docs/serverless/serverless-driver>
4. SQLite Documentation — <https://www.postgresql.org/docs/>
5. Custom CSS Documentation — <https://tailwindcss.com/docs>
6. Pressman, R. S. — Software Engineering: A Practitioner's Approach (McGraw-Hill)
7. Elmasri & Navathe — Fundamentals of Database Systems (Pearson)
8. IEEE Recommended Practice for Software Requirements Specifications — IEEE 830

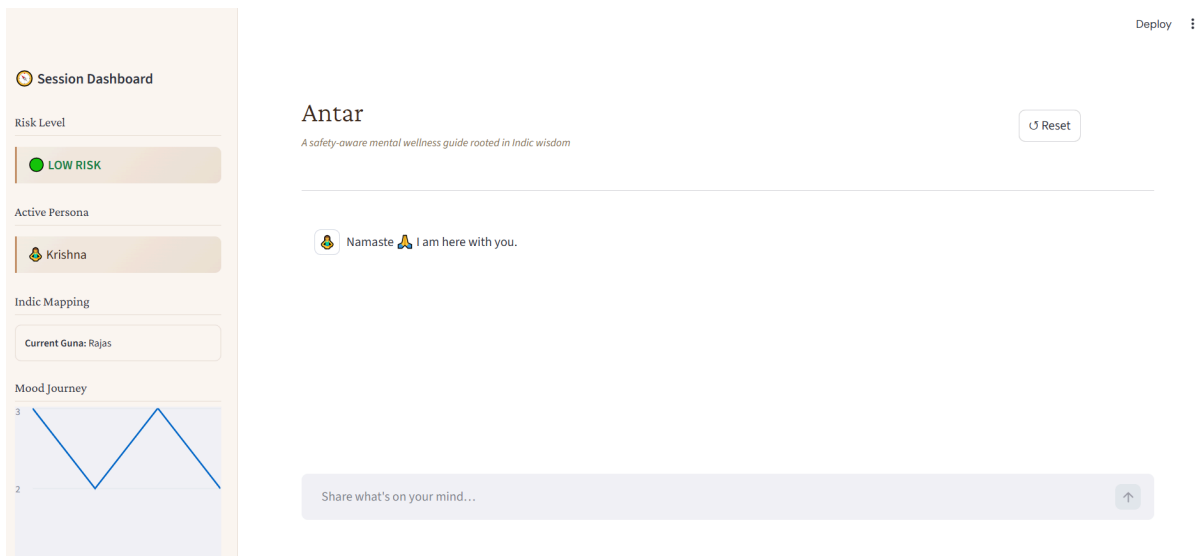
## Annexure 1: Input Forms with Data

The ANTAR system includes various input forms that allow users to interact with the application efficiently. These forms are designed using Streamlit and Custom CSS to provide a clean and user-friendly experience.

The following input forms are used in the system:



The image shows a 'Log In' form for the ANTAR system. The form is centered on a dark blue background. It includes a title 'Log In', a subtitle 'Enter your account information to proceed.', and a 'Forgot your password?' link. The form has two input fields: 'Email address' (with the placeholder 'you@example.com') and 'Password' (with masked characters '\*\*\*\*\*'). Below these fields is a 'Continue' button. Underneath the 'Continue' button is a section titled 'OR CONTINUE WITH' with three options: 'Continue with Google', 'Continue with GitHub', and 'Continue with Microsoft'. At the bottom of the form is a link 'New to Antar? Create an account -->'. The top left of the form has the ANTAR logo, and the top right has the text 'New to Antar? Create an account'.



The image shows the 'Session Dashboard' for the ANTAR system. The dashboard is divided into two main sections: a left sidebar and a main content area. The sidebar contains the following sections: 'Risk Level' (showing 'LOW RISK' with a green dot), 'Active Persona' (showing 'Krishna' with a person icon), 'Indic Mapping' (showing 'Current Guna: Rajas'), and 'Mood Journey' (showing a line graph with a peak and a trough). The main content area has a title 'Antar' and a subtitle 'A safety-aware mental wellness guide rooted in Indic wisdom'. Below the title is a 'Reset' button. Underneath the subtitle is a message 'Namaste 🙏 I am here with you.' with a person icon. At the bottom of the main content area is a text input field 'Share what's on your mind...' with a 'Send' button (represented by an upward arrow icon).



## Annexure 3: Sample Code

The following sample code snippets are used in the ANTAR project for implementing authentication, chat interaction, and database connectivity using Python and Streamlit.

---

### 1. Streamlit Application Setup

```
import streamlit as st
```

```
st.set_page_config(  
    page_title="ANTAR AI Assistant",  
    layout="wide"  
)
```

```
st.title("ANTAR AI Assistant")
```

Purpose:

- Initializes the Streamlit application.
  - Sets application title and layout.
- 

### 2. Login Authentication

```
email = st.text_input("Email")
```

```
password = st.text_input(  
    "Password",
```

```
        type="password"

    )

    if st.button("Login"):

        if email and password:

            st.success("Login Successful")
```

Purpose:

- Handles user login input.
  - Validates authentication fields.
- 

### **3. Chat Input System**

```
user_input = st.chat_input(

    "How are you feeling today?"

)

if user_input:

    st.chat_message("user").write(

        user_input

    )
```

```
response = "I am here to support you."
```

```
st.chat_message("assistant").write(  
  
    response  
  
)
```

Purpose:

- Allows users to interact with the AI assistant.
  - Displays chat conversation in real time.
- 

#### **4. SQLite Database Connection**

```
import sqlite3
```

```
def get_connection():
```

```
    return sqlite3.connect(  
  
        "antar.db"  
  
    )
```

Purpose:

- Establishes SQLite database connection.
  - Supports chat and user data storage.
-

## 5. Save Chat Messages

```
def save_chat_message(
```

```
    conversation_id,
```

```
    email,
```

```
    role,
```

```
    message
```

```
):
```

```
    conn = get_connection()
```

```
    cursor = conn.cursor()
```

```
    cursor.execute("""
```

```
INSERT INTO chats (
```

```
    conversation_id,
```

```
    email,
```

```
    role,
```

```
    message
```

```
) VALUES (?, ?, ?, ?)
```

```
""", (
```

```
    conversation_id,
```

```
    email,
```

```
    role,
```

```
    message
```

```
))
```

```
conn.commit()
```

```
conn.close()
```

Purpose:

- Stores chat messages in SQLite database.
- Maintains conversation history.

## **6. Create New Conversation**

```
def create_conversation(
```

```
    email,
```

```
    title="New Chat"
```

):

```
conn = get_connection()

cursor = conn.cursor()

cursor.execute("""

INSERT INTO conversations (

    email,

    title

) VALUES (?, ?)

""", (

    email,

    title

))

conn.commit()

conn.close()
```

Purpose:

- Creates new chat sessions for users.
- Supports multiple conversations.

---

## 7. Sidebar Navigation

with `st.sidebar`:

```
st.button("✚ New Chat")
```

```
st.markdown("### Chat History")
```

Purpose:

- Creates modern sidebar navigation.
  - Displays chat history and new chat options.
- 

## 8. Logout Functionality

```
def logout()
```

```
    st.session_state.clear()
```

```
    st.rerun()
```

Purpose:

- Clears user session.
- Logs user out securely.

[app.py](#)

```
"""
app.py - Streamlit UI for the Safety-aware Mental Wellness Chatbot.
"""

from dotenv import load_dotenv

load_dotenv()

import streamlit as st

from orchestrator import process_turn

from state_manager import initialize_state, reset_state

from utils import format_risk_badge

from database import initialize_database

from database import (
    initialize_database,

    save_chat_message,

    get_chat_history,

    save_mood,

    get_mood_history,
```



```
)

from auth import (

    initialize_auth,

    render_login_page,

    is_authenticated,

    logout,

)

from onboarding import (

    render_onboarding,

)

# -----

# PAGE CONFIG

# -----

st.set_page_config(

    page_title="Antar - Mental Wellness Guide",

    page_icon="🌸",
```

```
layout="wide",

initial_sidebar_state="expanded",
)

# -----

# DATABASE

# -----

from database import initialize_database

initialize_database()

# -----

# AUTH

# -----

initialize_auth()

if not is_authenticated():

    render_login_page()
```

```
st.stop()

# if not st.session_state.user_profile.get(

#     "onboarding_complete",

#     False

# ):

    render_onboarding()

    st.stop()

# -----

# SESSION STATE

# -----

if "chat_state" not in st.session_state:

    st.session_state.chat_state = initialize_state()

if "display_messages" not in st.session_state:
```

```
user_email = st.session_state.user_profile.get(

    "email",

    ""

)

if user_email:

    st.session_state.display_messages = get_chat_history(

        user_email

    )

else:

    st.session_state.display_messages = []

if "is_loading" not in st.session_state:

    st.session_state.is_loading = False

# -----

# CURRENT STATE
```

```
# -----

state = st.session_state.chat_state

risk = state.get(

    "risk_level",

    "LOW"

)

risk_class = f"risk-{risk.lower()}"

is_crisis_mode = risk == "HIGH"

# -----

# CSS

# -----

sidebar_bg = "#3b0a0a" if is_crisis_mode else "#fdf8f4"

sidebar_border = "#7f1d1d" if is_crisis_mode else "#e8ddd4"

st.markdown(f"""
```

```
<style>

@import
url('https://fonts.googleapis.com/css2?family=Crimson+Pro:ital,wght@0,300;0,400;1,300&family=DM+Sans:wght@300;400;500&display=swap');

html, body, [class*="css"] {{

    font-family: 'DM Sans', sans-serif;

}}

.main-title {{

    font-family: 'Crimson Pro', serif;

    font-size: 2.2rem;

    font-weight: 300;

    color: #3d2b1f;

}}

.subtitle {{
```

```
font-size: 0.85rem;

color: #8b7355;

font-style: italic;

margin-bottom: 1rem;
}}

.persona-badge {{

background: linear-gradient(

    135deg,

    #f5ede4 0%,

    #ede0d4 100%

);

border-left: 3px solid #c4956a;

border-radius: 0 8px 8px 0;
```

```
padding: 10px 14px;

margin: 8px 0;

color: #3d2b1f;
}}

.state-block {{

background: #faf7f4;

border: 1px solid #e8ddd4;

border-radius: 8px;

padding: 12px;

margin: 6px 0;

font-size: 0.80rem;
}}
```



```
.risk-high {{

    color: #c0392b;

    font-weight: 600;

}}

.risk-medium {{

    color: #d68910;

    font-weight: 600;

}}

.risk-low {{

    color: #1e8449;

    font-weight: 600;

}}

section[data-testid="stSidebar"] {{
```

```
background: {sidebar_bg};

border-right: 1px solid {sidebar_border};

}}
```

```
.sidebar-section-title {{

font-family: 'Crimson Pro', serif;

font-size: 1rem;

border-bottom: 1px solid #e8ddd4;

padding-bottom: 4px;

margin: 14px 0 6px 0;

}}
```

```
</style>
```

```
""", unsafe_allow_html=True)
```

```
# -----  
  
# HEADER  
  
# -----  
  
  
col_title, col_reset = st.columns([5, 1])  
  
  
with col_title:  
  
    st.markdown(  
  
        '<div class="main-title">Antar</div>',  
  
        unsafe_allow_html=True  
  
    )  
  
  
    st.markdown(  
  
        '<div class="subtitle">A safety-aware mental wellness guide  
rooted in Indic wisdom</div>',  
  
        unsafe_allow_html=True,  
  
    )  
  
  
with col_reset:  
  
  
    st.markdown("<br>", unsafe_allow_html=True)
```

```
if st.button("🔄 Reset"):  
  
    st.session_state.chat_state = reset_state()  
  
    st.session_state.display_messages = []  
  
    st.rerun()  
  
# -----  
# CRISIS MODE  
# -----  
  
if is_crisis_mode:  
  
    st.error(  
  
        "⚠️ Crisis Support Mode Activated"  
  
    )  
  
    st.markdown("""  
  
    <div style="
```

```
background:#fff5f5;

border:1px solid #fecaca;

padding:18px;

border-radius:12px;

margin-bottom:20px;

">

<h3 style="color:#991b1b;">

🧘 Take a Gentle Pause

</h3>

<p style="color:#7f1d1d; font-size:16px;">

• Inhale for 4 seconds<br>

• Hold for 4 seconds<br>

• Exhale for 6 seconds

<br><br>

Repeat slowly 3 times.

</p>
```

```
</div>

""" , unsafe_allow_html=True)

st.divider()

# -----
# SIDEBAR
# -----

with st.sidebar:

    st.markdown("### 🚨 Session Dashboard")

    # Risk

    st.markdown(

        '<div class="sidebar-section-title">Risk Level</div>',

        unsafe_allow_html=True

    )
```

```

st.markdown(

    f'<div class="persona-badge"><span
class="{risk_class}">{format_risk_badge(risk)}</span></div>',

    unsafe_allow_html=True,

)

# Persona

st.markdown(

    '<div class="sidebar-section-title">Active Persona</div>',

    unsafe_allow_html=True

)

st.markdown(

    f'<div class="persona-badge">🧘 {state.get("persona",
"Krishna")}</div>',

    unsafe_allow_html=True,

)

# Guna

st.markdown(

    '<div class="sidebar-section-title">Indic Mapping</div>',

```

```
        unsafe_allow_html=True

    )

    iks = state.get("iks", {})

    guna = (

        iks.get("guna")

        or iks.get("guna_state")

        or "Unknown"

    )

    st.markdown(

        f'<div class="state-block"><strong>Current Guna:</strong>
{guna}</div>',

        unsafe_allow_html=True

    )

    # -----

    # MOOD TRACKER

    # -----

    st.markdown(
```



```
        '<div class="sidebar-section-title">Mood Journey</div>',

        unsafe_allow_html=True

    )

    mood_rows = get_mood_history(

        st.session_state.user_profile.get(

            "email",

            ""

        )

    )

    if mood_rows:

        mapping = {

            "Tamas": 1,

            "Rajas": 2,

            "Sattva": 3,

        }
```

```
mood_scores = []

for row in mood_rows:

    mood_scores.append(

        mapping.get(

            row["guna"],

            2

        )

    )

st.line_chart(mood_scores)

st.markdown(

    f'<div class="state-block"><strong>Latest State:</strong>
{guna}</div>',

    unsafe_allow_html=True

)

else:
```

```

st.markdown(

    '<div class="state-block">No mood history yet.</div>',

    unsafe_allow_html=True

)

st.markdown("---")

if st.button(

    "🚪 Logout",

    use_container_width=True

):

    logout()

# -----

# CHAT HISTORY

# -----

for msg in st.session_state.display_messages:

    with st.chat_message(

```

```

        msg["role"],

        avatar="🧘" if msg["role"] == "assistant" else "😊"

    ):

        st.markdown(msg["content"])

# -----
# WELCOME
# -----

if not st.session_state.display_messages:

    with st.chat_message(

        "assistant",

        avatar="🧘"

    ):

        st.markdown(

            "Namaste 🙏 I am here with you."

        )

```

```
# -----  
  
# USER INPUT  
  
# -----  
  
user_input = st.chat_input(  
    "Share what's on your mind...",  
    disabled=st.session_state.is_loading,  
)  
  
# -----  
  
# MAIN FLOW  
  
# -----  
  
if user_input and user_input.strip():  
  
    # USER MESSAGE  
  
    st.session_state.display_messages.append({  
  
        "role": "user",
```

```
        "content": user_input

    })

    save_chat_message(

        st.session_state.user_profile.get(

            "email",

            ""

        ),

        "user",

        user_input

    )

    with st.chat_message(

        "user",

        avatar="😊"

    ):

        st.markdown(user_input)
```

```
# AI RESPONSE

with st.chat_message(

    "assistant",

    avatar="🧘 "

):

    with st.spinner("Reflecting..."):

        try:

            response, updated_state = process_turn(

                state=st.session_state.chat_state,

                user_message=user_input,

            )

            st.session_state.chat_state = updated_state

# SAVE MOOD
```

```
iks = updated_state.get(

    "iks",

    {}

)

guna = (

    iks.get("guna")

    or iks.get("guna_state")

)

if guna:

    save_mood(

        st.session_state.user_profile.get(

            "email",

            ""

        ),

        guna

    )
```



```
except Exception:

    response = (

        "I'm here with you. Something went wrong on my
end."

    )

st.markdown(response)

# SAVE RESPONSE

st.session_state.display_messages.append({

    "role": "assistant",

    "content": response

})

save_chat_message(

    st.session_state.user_profile.get(

        "email",
```

```
"""

    ),

    "assistant",

    response

)

st.rerun()
```

## [utility.py](#)

```
"""

utils.py - Advanced Utility Layer

FEATURES:

    ✓ Strong Hindi detection

    ✓ English detection

    ✓ Unicode-safe handling

    ✓ Smart JSON recovery

    ✓ Better language intelligence

"""

import json
```

```
import re

import unicodedata


# -----

# HINDI UNICODE RANGE

# -----


HINDI_CHAR_RANGE = r'[\u0900-\u097F]'


# -----

# HINDI WORDS

# -----


HINDI_WORDS = [

    "hai",

    "nahi",

    "kya",

    "kaise",

    "mujhe",

    "mera",

    "bahut",
```

```
"acha",

"thik",

"kyu",

"kar",

"karna",

"samajh",

"zindagi",

"dard",

"dukhi",

"akela",

"mann",

"dil",

]

# -----

# DETECT LANGUAGE

# -----

def detect_language(text: str) -> str:

    if not text:
```

```
        return "english"

text = normalize_text(text)

lower_text = text.lower()

# -----

# REAL HINDI SCRIPT

# -----

if re.search(HINDI_CHAR_RANGE, text):

    return "hindi"

# -----

# ROMAN HINDI DETECTION

# -----

hindi_matches = sum(

    1 for word in HINDI_WORDS
```

```
        if re.search(rf'\\b{word}\\b', lower_text)

    )

    # If enough Hindi words exist

    if hindi_matches >= 2:

        return "hindi"

    # Default

    return "english"

# -----
# NORMALIZE TEXT
# -----

def normalize_text(text: str) -> str:

    if not text:

        return ""
```

```
# Unicode normalization

text = unicodedata.normalize(

    "NFKC",

    text

)


# Remove extra spaces

text = re.sub(r"\s+", " ", text)

return text.strip()

# -----

# SANITIZE RESPONSE

# -----

def sanitize_response(text: str) -> str:

    if not text:
```

```
        return ""

    text = normalize_text(text)

    text = text.replace("`", "")

    text = text.replace("***", "")

    text = text.replace("###", "")

    return text.strip()

# -----
# SAFE JSON LOAD
# -----

def safe_json_load(
    raw_text: str,
    fallback=None
):

    if fallback is None:

        fallback = {}
```



```
if not raw_text:

    return fallback

raw_text = raw_text.strip()

raw_text = raw_text.replace(

    "`json",

    ""

)

raw_text = raw_text.replace(

    "`",

    ""

)

# Direct JSON parse

try:

    parsed = json.loads(raw_text)
```

```
        return repair_json_structure(parsed)

    except Exception:

        pass

    # Extract JSON block

    try:

        match = re.search(

            r'\{.*\}',

            raw_text,

            re.DOTALL

        )

        if match:

            extracted = match.group()

            parsed = json.loads(extracted)
```

```
        return repair_json_structure(parsed)

    except Exception:

        pass

    return repair_json_structure(fallback)

# -----
# REPAIR JSON STRUCTURE
# -----

def repair_json_structure(data: dict):

    if not isinstance(data, dict):

        return data

    # IKS compatibility

    if "guna_state" in data and "guna" not in data:
```

```
data["guna"] = data["guna_state"]
```

```
if "recommendation" not in data:
```

```
data["recommendation"] = ""
```

```
# Persona compatibility
```

```
if "persona" not in data:
```

```
data["persona"] = "Krishna"
```

```
if "reason" not in data:
```

```
data["reason"] = ""
```

```
return data
```

```
#
```

```
# WORD LIMIT
```

```
#
```

```
def truncate_to_word_limit(
```

```
    text: str,
```

```
    limit: int = 120
```

```
):
```

```
    if not text:
```

```
        return ""
```

```
    words = text.split()
```

```
    if len(words) <= limit:
```

```
        return text
```

```
    return " ".join(
```

```
        words[:limit]
```

```
    ) + "..."
```

```
#
```

---

```
# FALLBACKS
```

```
# -----  
  
def fallback_five_ps():  
  
    return {  
  
        "presenting": "",  
  
        "predisposing": "",  
  
        "precipitating": "",  
  
        "perpetuating": "",  
  
        "protective": "",  
    }  
  
# -----
```

```
def fallback_iks():  
  
    return {
```

```
        "guna": "rajas",

        "guna_state": "rajas",

        "recommendation":

            "grounding and reflection",

    }
```

```
#
```

---

```
def fallback_persona(current="Krishna"):
```

```
    return {

        "persona": current,

        "reason":

            "fallback persona selection",

    }
```

```
#
```

---

```
# CLEAN LLM OUTPUT
```

```
# _____
```

```
def clean_llm_output(text: str) -> str:
```

```
    text = sanitize_response(text)
```

```
    unwanted_prefixes = [
```

```
        "assistant:",
```

```
        "ai:",
```

```
        "bot:",
```

```
    ]
```

```
    lower = text.lower()
```

```
    for prefix in unwanted_prefixes:
```

```
        if lower.startswith(prefix):
```

```
            text = text[len(prefix):].strip()
```



```
    return text

# -----

# FORMAT RISK BADGE

# -----

def format_risk_badge(risk_level: str):

    risk_level = risk_level.upper()

    if risk_level == "HIGH":

        return "🔴 HIGH RISK"

    elif risk_level == "MEDIUM":

        return "🟡 MEDIUM RISK"

    return "🟢 LOW RISK"
```

### [database.py](#)

```
import sqlite3

from pathlib import Path
```

```
DB_NAME = "antar.db"
```

```
BASE_DIR = Path(__file__).resolve().parent
```

```
DATA_DIR = BASE_DIR / "data"
```

```
DATA_DIR.mkdir(exist_ok=True)
```

```
DB_PATH = DATA_DIR / "antar.db"
```

```
# _____
```

```
# CONNECTION
```

```
# _____
```

```
def get_connection():
```

```
    return sqlite3.connect(DB_PATH)
```

```
# _____
```

```
# INIT DATABASE
```

```
# _____
```

```
def initialize_database():

    conn = get_connection()

    cursor = conn.cursor()

    # USERS

    cursor.execute("""

CREATE TABLE IF NOT EXISTS users (

    id INTEGER PRIMARY KEY AUTOINCREMENT,

    email TEXT,

    name TEXT,

    age TEXT,

    gender TEXT,
```

```
        concern TEXT
    )
```

```
""")
```

```
# CHATS
```

```
cursor.execute("""
```

```
CREATE TABLE IF NOT EXISTS chats (
```

```
    id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    email TEXT,
```

```
    role TEXT,
```

```
    message TEXT
```

```
)
```

```
""")
```

```
# MOOD TRACKER

cursor.execute("""

CREATE TABLE IF NOT EXISTS mood_tracker (

    id INTEGER PRIMARY KEY AUTOINCREMENT,

    email TEXT,

    guna TEXT

)

""")

conn.commit()

conn.close()

# -----

# GET USER
```

```
#  
  
def get_user_by_email(email):  
  
    conn = get_connection()  
  
    conn.row_factory = sqlite3.Row  
  
    cursor = conn.cursor()  
  
    cursor.execute("""  
  
        SELECT *  
  
        FROM users  
  
        WHERE email = ?  
  
        """, (email,))  
  
    row = cursor.fetchone()
```

```
conn.close()

return dict(row) if row else None

# -----
# UPDATE USER PROFILE
# -----

def update_user_profile(

    email,

    name,

    age,

    gender,

    concern

):

    conn = get_connection()

    cursor = conn.cursor()

    existing = get_user_by_email(email)
```

```
if existing:

    cursor.execute("""

        UPDATE users

        SET

            name = ?,

            age = ?,

            gender = ?,

            concern = ?

        WHERE email = ?

    """, (

        name,

        age,

        gender,

        concern,
```



```
        email

    ))

else:

    cursor.execute("""

INSERT INTO users (

        email,

        name,

        age,

        gender,

        concern

    ) VALUES (?, ?, ?, ?, ?)

""", (

        email,

        name,

        age,
```

```
        gender,

        concern

    ))

conn.commit()

conn.close()

# -----
# SAVE CHAT
# -----

def save_chat_message(

    email,

    role,

    message

):

    conn = get_connection()

    cursor = conn.cursor()
```

```
cursor.execute("""
```

```
INSERT INTO chats (
```

```
    email,
```

```
    role,
```

```
    message
```

```
) VALUES (?, ?, ?)
```

```
""", (
```

```
    email,
```

```
    role,
```

```
    message
```

```
))
```

```
conn.commit()
```

```
conn.close()
```

```
# -----  
  
# GET CHAT HISTORY  
  
# -----  
  
def get_chat_history(email):  
  
    conn = get_connection()  
  
    conn.row_factory = sqlite3.Row  
  
    cursor = conn.cursor()  
  
    cursor.execute("""  
  
        SELECT role, message  
  
        FROM chats  
  
        WHERE email = ?  
  
        ORDER BY id ASC
```

```
""", (email,))
```

```
rows = cursor.fetchall()
```

```
conn.close()
```

```
history = []
```

```
for row in rows:
```

```
    history.append({
```

```
        "role": row["role"],
```

```
        "content": row["message"]
```

```
    })
```

```
return history
```

```
# _____
```

```
# SAVE MOOD
```

```
# _____
```

```
def save_mood(email, guna):

    conn = get_connection()

    cursor = conn.cursor()

    cursor.execute("""

INSERT INTO mood_tracker (

    email,

    guna

) VALUES (?, ?)

""", (

    email,

    guna

))
```

```
conn.commit()
```

```
conn.close()
```

```
# _____
```

```
# GET MOOD HISTORY
```

```
# _____
```

```
def get_mood_history(email):
```

```
    conn = get_connection()
```

```
    conn.row_factory = sqlite3.Row
```

```
    cursor = conn.cursor()
```

```
    cursor.execute("""
```

```
        SELECT *
```

```
        FROM mood_tracker
```

```
WHERE email = ?
```

```
ORDER BY id ASC
```

```
""", (email,))
```

```
rows = cursor.fetchall()
```

```
conn.close()
```

```
return rows
```

```
# _____
```

```
# GET USER PROFILE
```

```
# _____
```

```
def get_user_profile(email):
```

```
    conn = get_connection()
```

```
    conn.row_factory = sqlite3.Row
```



```
cursor = conn.cursor()
```

```
cursor.execute("""
```

```
    SELECT
```

```
        name,
```

```
        age,
```

```
        gender,
```

```
        concern
```

```
    FROM users
```

```
    WHERE email = ?
```

```
""", (email,))
```

```
row = cursor.fetchone()
```

```
conn.close()
```

```
if row:
```

```
        return {

            "name": row["name"],

            "age": row["age"],

            "gender": row["gender"],

            "concern": row["concern"],

            "onboarding_complete": True,

        }

    return None
```

## [auth.py](#)

```
"""

auth.py - Authentication UI for Antar

FIX: Only background ghost box removed. Login card stays white.

CSS specificity corrected - block-container rule is more specific than
wrapper rule.

"""
```

```
import streamlit as st

from streamlit_cookies_manager import EncryptedCookieManager

from database import get_user_profile


# -----

# COOKIE MANAGER

# -----

cookies = EncryptedCookieManager(

    prefix="antar_",

    password="antar_secure_password",

)

if not cookies.ready():

    st.stop()


# -----

# CSS

# -----

def _inject_css():
```

```

st.markdown("""

<style>

@import
url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap');

* { font-family: 'Inter', sans-serif !important; box-sizing:
border-box; }

/* — 1. Dark navy background — */

.stApp,

.stApp > div,

[data-testid="stAppViewContainer"],

[data-testid="stAppViewContainer"] > section,

[data-testid="stMain"] {

    background: linear-gradient(160deg, #020B3D 0%, #04196b 55%,
#0f2c96 100%) !important;

    min-height: 100vh !important;

}

/* — 2. Wrapper divs = transparent (kills ghost box)

    NOTE: .block-container rule below overrides this because

    it is written AFTER and uses a more specific selector — */

[data-testid="stMain"] > div,

[data-testid="stMain"] > div > div {

```

```
background: transparent !important;

box-shadow: none !important;

border-radius: 0 !important;

}
```

```
/* — 3. Login card – more specific selector so it wins over rule 2 — */
```

```
[data-testid="stMain"] div.block-container {

background: #ffffff !important;

border-radius: 20px !important;

box-shadow: 0 20px 80px rgba(0, 0, 0, 0.4) !important;

max-width: 460px !important;

width: 460px !important;

padding: 44px 44px 36px 44px !important;

margin: 72px auto 40px auto !important;

}
```

```
/* — 4. Hide Streamlit chrome + sidebar — */
```

```
[data-testid="stHeader"],

[data-testid="stToolbar"],

[data-testid="stDecoration"],

[data-testid="stStatusWidget"],
```

```
[data-testid="stSidebar"],

[data-testid="collapsedControl"],

#MainMenu, footer {

    display: none !important;

    width: 0 !important;

    height: 0 !important;

    opacity: 0 !important;

}


/* — 5. Brand logo top-left — */

.antar-brand {

    position: fixed;

    top: 22px;

    left: 32px;

    color: #ffffff;

    font-size: 22px;

    font-weight: 800;

    letter-spacing: 2.5px;

    z-index: 9999;

}


/* — 6. Nav top-right — */
```

```
.top-nav {  
  
    position: fixed;  
  
    top: 24px;  
  
    right: 32px;  
  
    color: rgba(255,255,255,0.75);  
  
    font-size: 13px;  
  
    z-index: 9999;  
  
}
```

```
/* — 7. Card title — */
```

```
.card-title {  
  
    font-size: 28px;  
  
    font-weight: 700;  
  
    color: #111827;  
  
    margin: 0 0 5px 0;  
  
}
```

```
.card-subtitle {  
  
    color: #6b7280;  
  
    font-size: 14px;  
  
    margin-bottom: 22px;  
  
    line-height: 1.5;  
  
}
```

```
/* — 8. Input labels — */

[data-testid="stTextInput"] label p {

    font-size: 13px !important;

    font-weight: 500 !important;

    color: #374151 !important;

}


/* — 9. Input fields — */

[data-testid="stTextInput"] input {

    border-radius: 10px !important;

    border: 1.5px solid #d1d5db !important;

    padding: 11px 14px !important;

    font-size: 14px !important;

    color: #111827 !important;

    background: #ffffff !important;

}

[data-testid="stTextInput"] input:focus {

    border-color: #c4956a !important;

    box-shadow: 0 0 0 3px rgba(196,149,106,0.18) !important;

    outline: none !important;

}
```



```
[data-testid="stTextInput"] input::placeholder { color: #9ca3af
!important; }

/* — Hide password reveal eye icon — */

[data-testid="stTextInput"] button {

    display: none !important;

}

/* — 10. Buttons — */

[data-testid="stButton"] > button {

    background: #c4956a !important;

    color: white !important;

    border-radius: 10px !important;

    border: none !important;

    height: 48px !important;

    font-weight: 600 !important;

    font-size: 15px !important;

    width: 100% !important;

    transition: background 0.2s !important;

    cursor: pointer !important;

}
```

```
[data-testid="stButton"] > button:hover { background: #a87850
!important; }
```

```
/* — 11. OR divider — */
```

```
.or-divider {
```

```
    display: flex;
```

```
    align-items: center;
```

```
    gap: 12px;
```

```
    margin: 20px 0;
```

```
    color: #9ca3af;
```

```
    font-size: 11px;
```

```
    font-weight: 600;
```

```
    letter-spacing: 1px;
```

```
}
```

```
.or-divider::before, .or-divider::after {
```

```
    content: '';
```

```
    flex: 1;
```

```
    height: 1px;
```

```
    background: #e5e7eb;
```

```
}
```

```
/* — 12. Social buttons — */
```

```
.social-btn {

    display: flex;

    align-items: center;

    gap: 12px;

    border: 1.5px solid #e5e7eb;

    border-radius: 10px;

    padding: 11px 16px;

    margin-bottom: 10px;

    font-weight: 500;

    font-size: 14px;

    color: #374151;

    background: white;

    cursor: pointer;

    transition: all 0.18s;

}

.social-btn:hover { background: #f9fafb; border-color: #c4956a; }

/* — 13. Forgot link — */

.forgot-link {

    color: #c4956a;

    font-size: 13px;

    font-weight: 500;
```

```
        text-align: right;

        margin: -2px 0 14px 0;

        display: block;

        cursor: pointer;

    }

/* — 14. Switch page text — */

.switch-text {

    text-align: center;

    color: #6b7280;

    font-size: 13px;

    margin-top: 12px;

}

.switch-text a { color: #c4956a; font-weight: 600; }

/* — 15. Password hint — */

.pw-hint { font-size: 11px; color: #9ca3af; margin: -6px 0 10px 0;
}

/* — 16. Alerts — */

[data-testid="stAlert"] { border-radius: 10px !important;
font-size: 13px !important; }
```

```

/* — 17. Tighten spacing — */

[data-testid="stVerticalBlock"] { gap: 0.3rem !important; }

</style>

<div class="antar-brand">🌸 ANTAR</div>

<div class="top-nav">New to Antar? Create an account</div>

""", unsafe_allow_html=True)

# _____

# SOCIAL BUTTONS

# _____

_SOCIAL_HTML = ""

<div class="or-divider">OR CONTINUE WITH</div>

<div class="social-btn">

    <svg width="18" height="18" viewBox="0 0 48 48">

        <path fill="#EA4335" d="M24 9.5c3.5 0 6.6 1.2 9.1
3.216.8-6.8C35.7 2.5 30.2 0 24 0 14.6 0 6.6 5.4 2.7 13.317.9 6.1C12.4
13 17.8 9.5 24 9.5z"/>

        <path fill="#4285F4" d="M46.1
24.6c0-1.6-.1-3.1-.4-4.6H24v8.7h12.4c-.5 2.9-2.2 5.4-4.7 717.3
5.7c4.3-3.9 6.8-9.7 6.8-16.8z"/>

```

```
<path fill="#FBBC05" d="M10.6
28.6c-.3-.9-.5-1.8-.5-2.8s.2-1.9.5-2.8l-7.9-6.1C1 18.9 0 21.4 0 24s1
5.1 2.7 7.117.9-6.1z"/>
```

```
<path fill="#34A853" d="M24 48c6.2 0 11.4-2
15.2-5.5l-7.3-5.7c-2 1.4-4.6 2.2-7.9 2.2-6.2 0-11.5-4.2-13.4-9.9l-7.9
6.1C6.6 42.6 14.6 48 24 48z"/>
```

```
</svg>
```

Continue with Google

```
</div>
```

```
<div class="social-btn">
```

```
<svg width="18" height="18" viewBox="0 0 24 24" fill="#111827">
```

```
<path d="M12 0C5.37 0 0 5.37 0 12c0 5.3 3.44 9.8 8.2
11.39.6.11.82-.26.82-.58v-2.04c-3.34.73-4.04-1.61-4.04-1.61-.55-1.39-1.
33-1.76-1.33-1.76-1.09-.74.08-.73.08-.73 1.2.09 1.84 1.24 1.84 1.24
1.07 1.83 2.8 1.3
3.49.99.11-.78.42-1.3.76-1.6-2.67-.3-5.47-1.33-5.47-5.93 0-1.31.47-2.38
1.24-3.22-.14-.3-.54-1.52.1-3.18 0 0 1-.32 3.3 1.23a11.5 11.5 0 0 1 6
0c2.28-1.55 3.29-1.23 3.29-1.23.64 1.66.24 2.88.12 3.18.77.84 1.23 1.91
1.23 3.22 0 4.61-2.81 5.63-5.48 5.92.43.37.81 1.1.81 2.22v3.29c0
.32.22.7.83.58C20.57 21.8 24 17.3 24 12c0-6.63-5.37-12-12-12z"/>
```

```
</svg>
```

Continue with GitHub

```
</div>
```

```
<div class="social-btn">
```

```
<svg width="18" height="18" viewBox="0 0 21 21">
```

```
<rect x="1" y="1" width="9" height="9" fill="#f25022"/>
```

```
<rect x="11" y="1" width="9" height="9" fill="#7fba00"/>
```

```
<rect x="1" y="11" width="9" height="9" fill="#00a4ef"/>
```

```

        <rect x="11" y="11" width="9" height="9" fill="#ffb900"/>

    </svg>

    Continue with Microsoft

</div>

"""

# -----
# INITIALIZE AUTH
# -----

def initialize_auth():

    if "authenticated" not in st.session_state:

        st.session_state.authenticated = (cookies.get("authenticated")
== "true")

    if "user_profile" not in st.session_state:

        st.session_state.user_profile = {

            "email":          cookies.get("email", ""),

            "name":           cookies.get("name", ""),

            "age":            cookies.get("age", ""),

            "gender":         cookies.get("gender", ""),

            "concern":        cookies.get("concern", ""),

```

```
        "onboarding_complete": cookies.get("onboarding_complete")
    == "true",

    }

    if "auth_page" not in st.session_state:

        st.session_state.auth_page = "login"


def is_authenticated():

    return st.session_state.get("authenticated", False)


# -----
# LOGIN PAGE
# -----


def render_login_page():

    _inject_css()

    st.markdown('<div class="card-title">Log In</div>',
unsafe_allow_html=True)

    st.markdown('<div class="card-subtitle">Enter your account
information to proceed.</div>', unsafe_allow_html=True)
```



```

    email    = st.text_input("Email address",
placeholder="you@example.com", key="login_email")

    password = st.text_input("Password", placeholder=".....",
key="login_pw", type="password")

    st.markdown('<div class="forgot-link">Forgot your password?</div>',
unsafe_allow_html=True)

    login_btn = st.button("Continue", use_container_width=True,
key="login_btn")

    st.markdown(_SOCIAL_HTML, unsafe_allow_html=True)

    st.markdown('<div class="switch-text">New to Antar? <a>Create an
account →</a></div>', unsafe_allow_html=True)

    # if st.button("Sign Up →", key="go_signup"):

    #     st.session_state.auth_page = "signup"

    #     st.rerun()

    if login_btn:

        if not email or not password:

            st.error("Please enter your email and password.")

            return

```

```
st.session_state.authenticated = True

cookies["authenticated"] = "true"

cookies["email"] = email

existing = get_user_profile(email)

if existing:

    st.session_state.user_profile = {

        "email": email, "name": existing.get("name", ""),

        "age": existing.get("age", ""), "gender":
existing.get("gender", ""),

        "concern": existing.get("concern", ""),
"onboarding_complete": True,

    }

    cookies["name"] = existing.get("name", "")

    cookies["age"] = str(existing.get("age", ""))

    cookies["gender"] = existing.get("gender", "")

    cookies["concern"] = existing.get("concern", "")

    cookies["onboarding_complete"] = "true"

else:

    st.session_state.user_profile = {

        "email": email, "name": "", "age": "",

        "gender": "", "concern": "", "onboarding_complete":
False,

    }
```

```
cookies.save()

st.rerun()

# -----

# SIGNUP PAGE

# -----

# def render_signup_page():

#     _inject_css()

#     st.markdown('<div class="card-title">Create Account</div>',
unsafe_allow_html=True)

#     st.markdown('<div class="card-subtitle">Start your wellness
journey with Antar today.</div>', unsafe_allow_html=True)

#     name      = st.text_input("Full Name",          placeholder="Your
full name",      key="su_name")

#     email     = st.text_input("Email address",
placeholder="you@example.com",    key="su_email")

#     pwd       = st.text_input("Password",          placeholder="Min. 8
characters", key="su_pwd",    type="password")

#     st.markdown('<div class="pw-hint">Use 8+ characters with letters
and numbers.</div>', unsafe_allow_html=True)
```

```
#     confirm = st.text_input("Confirm Password", placeholder="Re-enter
password", key="su_conf", type="password")

#     signup_btn = st.button("Create Account",
use_container_width=True, key="signup_btn")

#     st.markdown(_SOCIAL_HTML, unsafe_allow_html=True)

#     st.markdown('<div class="switch-text">Already have an account?
<a>Sign in</a></div>', unsafe_allow_html=True)

#     if st.button("← Back to Login", key="go_login"):

#         st.session_state.auth_page = "login"

#         st.rerun()

#     if signup_btn:

#         if not name or not email or not pwd or not confirm:

#             st.error("Please fill in all fields.")

#         elif len(pwd) < 8:

#             st.error("Password must be at least 8 characters.")

#         elif pwd != confirm:

#             st.error("Passwords do not match.")

#         else:
```

```

#         st.session_state.authenticated = True

#         cookies["authenticated"]          = "true"

#         cookies["email"]                  = email

#         cookies["name"]                   = name

#         cookies["onboarding_complete"] = "false"

#         st.session_state.user_profile = {

#             "email": email, "name": name, "age": "",

#             "gender": "", "concern": "", "onboarding_complete":

False,

#             }

#         cookies.save()

#         st.success(f"Welcome to Antar, {name}! 🌸")

#         st.rerun()

# -----

# MAIN ENTRY POINT

# -----

# def render_auth_page():

#     if st.session_state.get("auth_page") == "signup":

#         render_signup_page()

```

```

#         else:

#             render_login_page()

def save_onboarding_cookies(profile):

    cookies["name"] = str(profile.get("name", ""))

    cookies["age"] = str(profile.get("age", ""))

    cookies["gender"] = str(profile.get("gender", ""))

    cookies["concern"] = str(profile.get("concern", ""))

    cookies["onboarding_complete"] = "true"

    cookies.save()

def logout():

    cookies["authenticated"] = "false"

    cookies.save()

    for key in list(st.session_state.keys()):

        del st.session_state[key]

    st.rerun()

```

[onborinding.py](#)

```
"""
onboarding.py - Antar onboarding form
"""

import streamlit as st

from database import update_user_profile

from auth import save_onboarding_cookies


# -----
# ONBOARDING FORM
# -----


def render_onboarding():

    st.markdown("""

<style>

.stApp {

    background: #020b3d;
```

```
}

.onboarding-card {

  background: white;

  width: 500px;

  margin: auto;

  margin-top: 40px;

  padding: 30px;

  border-radius: 12px;

  box-shadow: 0 10px 30px rgba(0,0,0,0.25);
}

.title {

  font-size: 30px;
```



```
        font-weight: 700;

        color: #111827;

        margin-bottom: 6px;
    }

    .subtitle {

        color: #6b7280;

        margin-bottom: 24px;
    }

</style>

""", unsafe_allow_html=True)

st.markdown(

    '<div class="onboarding-card">',

    unsafe_allow_html=True
```

```
)

st.markdown(

    '<div class="title">Welcome to Antar</div>',

    unsafe_allow_html=True

)

st.markdown(

    '<div class="subtitle">Help us personalize your  
experience.</div>',

    unsafe_allow_html=True

)

name = st.text_input(

    "Your Name"

)

age = st.number_input(

    "Your Age",

    min_value=10,

    max_value=100,

    step=1
```

```
)

gender = st.selectbox(

    "Gender",

    [

        "Male",

        "Female",

        "Other",

        "Prefer not to say",

    ]

)

concern = st.text_area(

    "What brings you here today?"

)

continue_btn = st.button(

    "Continue to Antar",

    use_container_width=True

)

if continue_btn:
```

```
if name:

    # Update database

    update_user_profile(

        st.session_state.user_profile.get(

            "email",

            ""

        ),

        name,

        age,

        gender,

        concern

    )

    # Update session

    st.session_state.user_profile.update({
```

```
        "name": name,

        "age": age,

        "gender": gender,

        "concern": concern,

        "onboarding_complete": True,
    })

    # SAVE COOKIES

    save_onboarding_cookies(

        st.session_state.user_profile

    )

    st.rerun()

else:

    st.error(
```

```
        "Please enter your name."

    )

    st.markdown(

        '</div>',

        unsafe_allow_html=True

    )
```

## [risk.py](#)

```
"""

risk.py - Advanced LLM-Based Risk Detection

FEATURES:

    ✓ English support

    ✓ Hindi support

    ✓ Semantic risk analysis

    ✓ LLM-based emotional understanding

    ✓ Confidence scoring

    ✓ Emergency escalation

"""

import re

import json
```

```
import logging

import os

from dotenv import load_dotenv

from groq import Groq

load_dotenv()

# -----

# LOGGING

# -----

logger = logging.getLogger(__name__)

# -----

# GROQ CLIENT

# -----

client = Groq(

    api_key=os.getenv("GROQ_API_KEY")

)
```

```
#  
  
# FAST EMERGENCY KEYWORDS
```

```
#  
  
EMERGENCY_PATTERNS = [  
  
    # English
```

```
    r"kill myself",  
  
    r"suicide",  
  
    r"want to die",  
  
    r"end my life",
```

```
    # Hindi  
  
    r"मरना चाहता",  
  
    r"जीना नहीं",
```

```
    # Roman Hindi  
  
    r"marna chahta",  
  
    r"jeena nahi",
```

```
]
```

```
#
```



```
# NORMALIZE TEXT

# -----

def normalize_text(text: str) -> str:

    if not text:

        return ""

    text = text.lower()

    text = re.sub(r"\s+", " ", text)

    return text.strip()

# -----

# QUICK CHECK

# -----

def quick_risk_check(text: str) -> bool:

    cleaned = normalize_text(text)
```

```
for pattern in EMERGENCY_PATTERNS:

    if re.search(pattern, cleaned):

        logger.warning(

            f"[Emergency] matched: {pattern}"

        )

        return True

return False

# -----
# LLM SEMANTIC ANALYSIS
# -----

def llm_risk_analysis(text: str):

    prompt = f"""

    You are a mental health risk classifier.
```

Analyze the emotional risk level  
of the following user message.

Consider:

- suicidal intent
- hopelessness
- emotional collapse
- self-harm risk
- severe distress
- panic
- depression

Understand:

- English
- Hindi
- Roman Hindi

Return ONLY valid JSON.

RISK LEVELS:

- LOW

```
- MEDIUM
```

```
- HIGH
```

```
JSON FORMAT:
```

```
{{  
  
  "risk_level": "",  
  
  "confidence": 0.0,  
  
  "reason": ""  
}}
```

```
USER MESSAGE:
```

```
{text}
```

```
"""
```

```
try:
```

```
    response = client.chat.completions.create(  
  

```

```
        model="llama-3.1-8b-instant",  

```

```
        messages=[

            {

                "role": "user",

                "content": prompt

            }

        ],

        max_tokens=120,

        temperature=0.2,

    )

    raw = response.choices[

        0

    ].message.content.strip()

    logger.info(

        f"[LLM Risk Raw] {raw}"

    )

    # Clean markdown
```

```
raw = raw.replace(

    "```json",

    ""

)

raw = raw.replace(

    "```",

    ""

)

result = json.loads(raw)

return result

except Exception as e:

    logger.error(

        f"[LLM Risk Error] {e}"

    )

return {
```

```
        "risk_level": "LOW",

        "confidence": 0.2,

        "reason": "fallback",
    }

# -----

# MAIN RISK DETECTION

# -----

def detect_risk(text: str) -> str:

    result = analyze_risk(text)

    return result["risk_level"]

# -----

# FULL ANALYSIS

# -----

def analyze_risk(text: str):
```

```
# Fast emergency bypass

if quick_risk_check(text):

    return {

        "risk_level": "HIGH",

        "confidence": 0.99,

        "reason": "emergency keyword",

    }

# LLM semantic analysis

llm_result = llm_risk_analysis(text)

logger.info(

    f"[Semantic Risk] {llm_result}"

)
```



```
        return llm_result

# -----

# ESCALATION RESPONSE

# -----
```

```
def escalation_response() -> str:
```

```
    return """
```

I'm really glad you shared this.

You do not have to go through this alone.

Please consider reaching out to:

- A trusted friend
- Family member
- Mental health professional

अगर आप तुरंत खतरे में हैं,

कृपया किसी भरोसेमंद व्यक्ति से संपर्क करें।

```
'''
```

[state.py](#)

```
from copy import deepcopy

from datetime import datetime
```

```
# -----
```

```
# CONFIG
```

```
# -----
```

```
MAX_HISTORY_MESSAGES = 20
```

```
MAX_CONTEXT_MESSAGES = 10
```

```
MAX_GUNA_HISTORY = 50
```

```
# -----
```

```
# INITIAL STATE
```

```
# -----
```

```
def create_initial_state():
```

```
    return {
```

```
# Conversation

"messages": [],

# Language

"language": "english",

# Persona

"persona": "Krishna",

"persona_reason": "",

# Risk

"risk_level": "LOW",

# Emotional trends

"emotion_history": [],

# NEW: Mood / Guna history

"guna_history": [],

# Psychological formulation

"five_ps": {
```

```
        "presenting": "",

        "predisposing": "",

        "precipitating": "",

        "perpetuating": "",

        "protective": "",
    },

    # Indic Knowledge System

    "iks": {

        "guna": "Rajas",

        "guna_state": "Rajas",

        "recommendation": "grounding and reflection",
    },
```

```
# Analytics

"analytics": {

    "total_messages": 0,

    "user_messages": 0,

    "assistant_messages": 0,

    "session_start": str(datetime.utcnow()),

    "last_updated": str(datetime.utcnow()),

},

}

# -----

# BACKWARD COMPATIBILITY

# -----

def initialize_state():
```

```
    return create_initial_state()

# -----

# SAFE COPY

# -----

def safe_state_copy(state):

    return deepcopy(state)

# -----

# TIMESTAMP

# -----

def current_timestamp():

    return datetime.utcnow().isoformat()

# -----

# APPEND MESSAGE
```

```
# -----  
  
def append_message(state, role, content):  
  
    state = safe_state_copy(state)  
  
    message = {  
  
        "role": role,  
  
        "content": str(content),  
  
        "timestamp": current_timestamp(),  
    }  
  
    state["messages"].append(message)  
  
    # Analytics  
  
    state["analytics"]["total_messages"] += 1
```

```
if role == "user":

    state["analytics"]["user_messages"] += 1

elif role == "assistant":

    state["analytics"]["assistant_messages"] += 1

state["analytics"]["last_updated"] = current_timestamp()

# Auto trim

state = trim_conversation(state)

return state

# -----
# TRIM CONVERSATION
# -----

def trim_conversation(state):
```



```
state = safe_state_copy(state)
```

```
if len(state["messages"]) > MAX_HISTORY_MESSAGES:
```

```
    state["messages"] = state["messages"][-MAX_HISTORY_MESSAGES:]
```

```
return state
```

```
# _____
```

```
# GET HISTORY
```

```
# _____
```

```
def get_conversation_history(state):
```

```
    messages = state.get("messages", [])
```

```
    recent_messages = messages[-MAX_CONTEXT_MESSAGES:]
```

```
    formatted_history = []
```

```
for msg in recent_messages:

    role = msg.get("role", "unknown")

    content = msg.get("content", "")

    formatted_history.append(

        f"{role}: {content}"

    )

    return "\n".join(formatted_history)
```

```
# -----
# UPDATE FIVE PS
# -----
```

```
def update_five_ps(state, new_five_ps):

    state = safe_state_copy(state)

    state["five_ps"] = new_five_ps
```

```
        return state

# -----

# UPDATE IKS + GUNA HISTORY

# -----


def update_iks(state, new_iks):

    state = safe_state_copy(state)

    state["iks"] = new_iks

    # Detect guna safely

    guna = (

        new_iks.get("guna")

        or new_iks.get("guna_state")

        or "Unknown"

    )
```

```
# Save guna history

state["guna_history"].append({

    "guna": guna,

    "timestamp": current_timestamp(),

})

# Keep latest entries only

state["guna_history"] = state[

    "guna_history"

][-MAX_GUNA_HISTORY:]

return state

# -----

# UPDATE PERSONA

# -----
```

```
def update_persona(state, persona, reason=""):
```

```
    state = safe_state_copy(state)
```

```
    state["persona"] = persona
```

```
    state["persona_reason"] = reason
```

```
    return state
```

```
#
```

```
# UPDATE RISK
```

```
#
```

```
def update_risk(state, risk_level):
```

```
    state = safe_state_copy(state)
```

```
    state["risk_level"] = risk_level
```

```
    state["emotion_history"].append({
```

```

        "risk_level": risk_level,

        "timestamp": current_timestamp(),

    })

    # Keep latest entries only

    state["emotion_history"] = state[

        "emotion_history"

    ][-30:]

    return state

# -----

# VALIDATION

# -----

def validate_state(state):

    required_keys = [

```

```
        "messages",

        "language",

        "persona",

        "risk_level",

        "five_ps",

        "iks",

        "analytics",

    ]
```

```
    for key in required_keys:
```

```
        if key not in state:
```

```
            return False
```

```
    return True
```

```
# -----
# RESET STATE
# -----
```

```
def reset_state():  
  
    return create_initial_state()
```

```
# -----
```

```
# EXPORT STATE
```

```
# -----
```

```
def export_state(state):  
  
    return safe_state_copy(state)
```

```
# -----
```

```
# GUNA ANALYTICS
```

```
# -----
```

```
def get_guna_counts(state):  
  
    guna_history = state.get(
```



```
        "guna_history",

        []

    )

counts = {

    "Sattva": 0,

    "Rajas": 0,

    "Tamas": 0,

}

for item in guna_history:

    guna = item.get("guna", "")

    if guna in counts:

        counts[guna] += 1

return counts
```

```
#
```

---

```
# SESSION SUMMARY
```

```
# -----  
  
def get_session_summary(state):  
  
    analytics = state.get("analytics", {})  
  
    return {  
  
        "total_messages":  
            analytics.get("total_messages", 0),  
  
        "user_messages":  
            analytics.get("user_messages", 0),  
  
        "assistant_messages":  
            analytics.get("assistant_messages", 0),  
  
        "current_persona":  
            state.get("persona", "Unknown"),  
  
        "risk_level":
```

```
        state.get("risk_level", "LOW"),

        "language":

        state.get("language", "english"),

    }
```

### [pormpotys.py](#)

```
"""

prompts.py - Advanced Prompt Engineering Layer


FEATURES:

    ✓ Hindi support

    ✓ Hinglish support

    ✓ Better empathy

    ✓ Persona consistency

    ✓ Structured prompting

    ✓ Hallucination reduction

    ✓ Safer emotional responses

"""

# -----

# SYSTEM SAFETY RULES
```

```
# -----  
  
SAFETY_RULES = """  
  
IMPORTANT RULES:  
  
1. Never encourage self-harm.  
2. Never shame the user.  
3. Never give dangerous advice.  
4. Stay emotionally supportive.  
5. Keep responses warm and human.  
6. Avoid robotic language.  
7. Keep responses concise.  
8. Never act like a doctor.  
9. Never diagnose mental illness.  
10. If user is high-risk:  
    encourage human support gently.  
  
"""
```

```
# -----  
  
# LANGUAGE STYLE
```

```
# _____
```

```
LANGUAGE_STYLES = {
```

```
    "english": ""
```

```
Respond in natural conversational English.
```

```
Style:
```

- warm
- calm
- emotionally intelligent
- short paragraphs
- supportive
- non-robotic

```
"" ,
```

```
    "hinglish": ""
```

```
Respond in natural Hinglish.
```

Rules:

- Use casual Indian conversational tone
- Mix Hindi + English naturally
- Avoid overly formal Hindi
- Sound emotionally supportive
- Keep responses human and warm

Example tone:

"Main samajh sakta hun ye kitna heavy feel ho sakta hai."

"" ,

"hindi": ""

उत्तर पूरी तरह प्राकृतिक हिन्दी में दें।

नियम:

- भाषा सरल और मानवीय हो
- बहुत अधिक संस्कृतनिष्ठ शब्द न हों
- भावनात्मक रूप से सहायक रहें
- उत्तर छोटे और स्पष्ट हों

उदाहरण शैली:

"मैं आपकी बात समझ रहा हूँ।"

"""

}

# \_\_\_\_\_

# PERSONA STYLES

# \_\_\_\_\_

PERSONA\_STYLES = {

    "Krishna": """

Persona Style:

- calm wisdom
- compassionate guidance
- emotionally balanced
- grounding perspective

""",

"Buddha": ""

Persona Style:

- peaceful
- mindful
- reflective
- gentle emotional clarity

"" ,

"Patanjali": ""

Persona Style:

- disciplined
- grounding
- breath-focused
- clarity and balance

"" ,

"Upanishadic Guide": ""



```
Persona Style:
```

- introspective
- philosophical
- emotionally deep
- calm and thoughtful

```
""",
```

```
}
```

```
# _____
```

```
# FIVE PS PROMPT
```

```
# _____
```

```
def build_fiveps_update_prompt(
```

```
    user_message,
```

```
    previous_five_ps,
```

```
    conversation_history
```

```
):
```

```
    return f"""
```

```
You are a psychological formulation assistant.
```

Your task:

Update the user's 5Ps psychological model.

5Ps:

1. Presenting
2. Predisposing
3. Precipitating
4. Perpetuating
5. Protective

Return ONLY valid JSON.

Previous 5Ps:

```
{previous_five_ps}
```

Conversation History:

```
{conversation_history}
```

User Message:

```
{user_message}
```

JSON FORMAT:

```
{{
    "presenting": "",
    "predisposing": "",
    "precipitating": "",
    "perpetuating": "",
    "protective": ""
}}
```

"""

#

# IKS PROMPT

#

```
def build_iks_prompt(five_ps):
```

```
    return f"""
```

You are an Indic psychology assistant.

Analyze the psychological state using:

- Sattva
- Rajas
- Tamas

Return ONLY valid JSON.

5Ps:

```
{five_ps}
```

JSON FORMAT:

```
{{  
  
  "guna_state": "",  
  
  "recommendation": ""  
}}
```

"""

```
# _____
```

```
# PERSONA SELECTION
```

```
# _____
```

```
def build_persona_prompt(
```

```
    five_ps,
```

```
    iks,
```

```
    current_persona
```

```
):
```

```
    return f"""
```

You are a persona selection engine.

Available personas:

- Krishna
- Buddha
- Patanjali
- Upanishadic Guide

Task:

Select the BEST persona

for the user's current emotional state.

Rules:

- Return ONLY valid JSON
- Keep reasoning short

Current Persona:

```
{current_persona}
```

5Ps:

```
{five_ps}
```

IKS:

```
{iks}
```

JSON FORMAT:

```
{{  
  
  "persona": "",  
  
  "reason": ""  
  
}}
```

"""

# \_\_\_\_\_

```
# RESPONSE PROMPT
```

```
# _____
```

```
def build_response_prompt(
```

```
    user_message,
```

```
    persona,
```

```
    five_ps,
```

```
    iks,
```

```
    conversation_history,
```

```
    language="english"
```

```
):
```

```
    language_style = LANGUAGE_STYLES.get(
```

```
        language,
```

```
        LANGUAGE_STYLES["english"]
```

```
    )
```

```
    persona_style = PERSONA_STYLES.get(
```

```
        persona,
```

```
        PERSONA_STYLES["Krishna"]
```

```
    )
```

```
return f"""
```

You are Antar,

an emotionally supportive AI wellness companion.

{SAFETY\_RULES}

{language\_style}

{persona\_style}

Conversation History:

{conversation\_history}

Psychological Context:

{five\_ps}

Indic Context:

{iks}

Current User Message:

{user\_message}



IMPORTANT RESPONSE RULES:

1. Respond naturally.
2. Be emotionally supportive.
3. Keep response under 120 words.
4. Avoid sounding robotic.
5. Avoid repeating the user.
6. Do not lecture.
7. Do not over-explain.
8. Be calm and grounded.
9. Validate emotions gently.
10. Use conversational tone.

Generate the response now.

"""

#

# ESCALATION PROMPT

#

```
def build_escalation_prompt():  
  
    return """  
  
You are responding to a high-risk emotional situation.  
  
Rules:  
  
- Stay calm  
  
- Be supportive  
  
- Encourage immediate human support  
  
- Avoid panic language  
  
- Keep response gentle and direct  
  
"""
```

## [README.md](#)

```
# 🌸 Antar – Safety-Aware Mental Wellness Chatbot
```

```
> "Antar" (अन्तर) – Sanskrit for "inner" or "within".
```

A modular, safety-aware mental wellness chatbot grounded in **Indic Knowledge Systems (IKS)**,

powered by dynamic **5Ps psychological formulation**, and delivered through persona-adaptive

response generation using Claude (Anthropic).

---

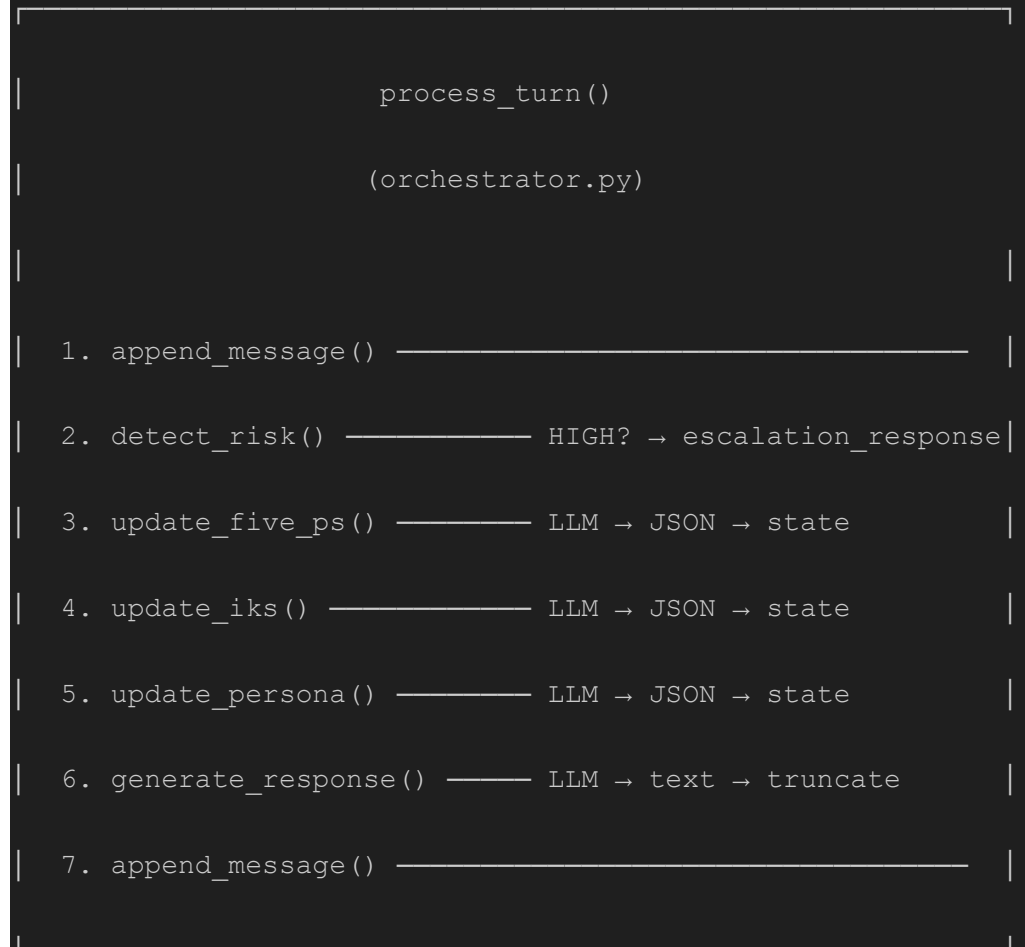
## ## Architecture Overview

...

User Input

|

▼



Streamlit UI (app.py)

└─ Chat messages

└─ Sidebar: risk | persona | IKS | 5Ps | raw JSON

...

### ### Module Responsibilities

| File | Responsibility |
|------|----------------|
|------|----------------|

| --- | --- |
|-----|-----|
|-----|-----|

|          |                                                      |
|----------|------------------------------------------------------|
| `app.py` | Streamlit UI, session state, chat interface, sidebar |
|----------|------------------------------------------------------|

|                   |                                                   |
|-------------------|---------------------------------------------------|
| `orchestrator.py` | Pipeline controller, LLM calls, step coordination |
|-------------------|---------------------------------------------------|

|              |                                                       |
|--------------|-------------------------------------------------------|
| `prompts.py` | All LLM prompt builders (5Ps, IKS, persona, response) |
|--------------|-------------------------------------------------------|

|           |                                                   |
|-----------|---------------------------------------------------|
| `risk.py` | Keyword-based risk detection, escalation response |
|-----------|---------------------------------------------------|

|                    |                                                  |
|--------------------|--------------------------------------------------|
| `state_manager.py` | Immutable state operations (init, update, reset) |
|--------------------|--------------------------------------------------|

|            |                                                 |
|------------|-------------------------------------------------|
| `utils.py` | Safe JSON parsing, truncation, fallback helpers |
|------------|-------------------------------------------------|

|             |                                              |
|-------------|----------------------------------------------|
| `config.py` | Personas, keywords, defaults, model settings |
|-------------|----------------------------------------------|

|                   |                                                      |
|-------------------|------------------------------------------------------|
| `test_cases.json` | 10 annotated test cases for manual/automated testing |
|-------------------|------------------------------------------------------|

```
---

## Setup Instructions
```

```
### 1. Prerequisites
```

- Python 3.11+
- An **Anthropic API key** ([get one here](https://console.anthropic.com/))

```
### 2. Clone / copy the project
```

```
```bash

mkdir antar && cd antar

# copy all project files here

```
```

```
### 3. Create a virtual environment
```

```
```bash

python -m venv .venv

source .venv/bin/activate      # Linux/macOS

.venv\Scripts\activate        # Windows

```
```

### ### 4. Install dependencies

```
```bash  
  
pip install -r requirements.txt  
  
```
```

### ### 5. Set your API key

Create a `.env` file in the project root:

```
```  
  
ANTHROPIC_API_KEY=sk-ant-...your-key-here...  
  
```
```

Or export it directly:

```
```bash  
  
export ANTHROPIC_API_KEY="sk-ant-..."  
  
```
```

### ### 6. Run the app

```
```bash  
  
streamlit run app.py  
  
```
```

The app will open at `http://localhost:8501`.

---

## ## Features

### ### 🛡️ Safety-First Design

- Keyword-based HIGH/MEDIUM/LOW risk classification on every message
- HIGH risk (suicidal ideation, self-harm) **\*\*immediately bypasses all pipelines\*\***  
  
and returns a compassionate crisis escalation message with helpline numbers
- No persona or psychological analysis is performed on HIGH-risk inputs

### ### 🧠 Dynamic 5Ps Formulation

After every user message, the LLM updates all five dimensions:

- **\*\*Presenting Problem\*\*** - What is distressing the person right now
- **\*\*Predisposing Factors\*\*** - Background vulnerabilities
- **\*\*Precipitating Factors\*\*** - Recent triggers
- **\*\*Perpetuating Factors\*\*** - What is maintaining the distress
- **\*\*Protective Factors\*\*** - Strengths and resources

### ### 🌟 Indic Knowledge Mapping

Maps the formulation to:

- **Guna** (Sattva / Rajas / Tamas) – Quality of mental state
- **Dominant Concepts** – From attachment, chitta vritti, dharma conflict, viveka, etc.
- **Brief Mapping** – Explanation of how IKS concepts apply

### ### 🧑🏽 Persona-Adaptive Responses

Four Indic wisdom personas, auto-selected each turn:

| Persona                  | Best For                                       |
|--------------------------|------------------------------------------------|
| ---                      | ---                                            |
| <b>Krishna</b>           | Dharma conflict, identity, purpose, anger      |
| <b>Patanjali</b>         | Anxiety, overthinking, mental agitation        |
| <b>Buddha</b>            | Grief, sadness, attachment, impermanence       |
| <b>Upanishadic Guide</b> | Existential questions, self-inquiry, emptiness |

### ### 📊 Live Sidebar Dashboard

- Risk level badge (🟢/🟡/🔴)
- Current persona + reasoning
- IKS guna + concepts + mapping
- 5Ps formulation (expandable)
- Raw JSON state viewer

---



## ## How to Run Test Cases

```
```bash

python - <<'EOF'

import json

from orchestrator import process_turn

from state_manager import initialize_state


with open("test_cases.json") as f:

    cases = json.load(f)


for case in cases:

    state = initialize_state()

    print(f"\n--- Test {case['id']}: {case['category']} ---")

    response, state = process_turn(state, case["input"])

    print(f"Risk: {state['risk_level']} (expected:
{case['expected_risk']})")

    print(f"Persona: {state['persona']} (expected:
{case['likely_persona']})")

    print(f"Guna: {state['iks']['guna']}")

    print(f"Response: {response[:120]}...")

EOF
```

## ## Limitations

1. **\*\*Not a clinical tool\*\*** – This chatbot is not a replacement for professional mental health care.

It should not be used for diagnosis, treatment, or crisis intervention beyond signposting.

2. **\*\*Keyword-only risk detection\*\*** – The risk engine uses pattern matching, not clinical assessment.

It can produce false positives (over-sensitive) and false negatives (missed signals).

3. **\*\*LLM reliability\*\*** – JSON outputs from intermediate steps may occasionally fail to parse;

fallback values are applied silently in these cases.

4. **\*\*Cultural context\*\*** – IKS mappings and persona voices are approximate; they are inspired by

the traditions but do not represent authoritative scholarly interpretations.

# CHAPTER 1: INTRODUCTION

ANTAR is an AI-powered Mental Wellness Assistant developed using Python, Streamlit, SQLite, and Groq Llama AI integration. The main objective of the project is to provide users with intelligent and supportive AI conversations through a modern web-based platform.

The system allows users to securely log in, create conversations, access previous chat history, and interact with an AI assistant using a responsive ChatGPT-style interface. The application focuses on improving user experience by providing secure authentication, persistent sessions, modern sidebar navigation, and conversation management.

The ANTAR platform demonstrates the practical implementation of frontend development, backend processing, AI integration, database management, and responsive UI design using modern Python technologies.

---

## CHAPTER 2: PROPOSED SYSTEM

The proposed ANTAR system is a web-based AI Mental Wellness Assistant platform designed to provide users with intelligent AI interaction and conversation support in a secure environment.

The system is developed using:

- Python
- Streamlit
- SQLite
- Groq Llama AI
- Custom CSS

The application provides:

- Secure authentication system
- AI-powered chat assistant
- Persistent conversation history
- Multiple chat sessions
- Responsive sidebar layout
- Modern user interface

The system architecture includes frontend UI, backend logic, AI integration, database management, and session handling modules working together to provide smooth user interaction.

---

## 3.6 Sample Code

```
import streamlit as st
```

```
st.title("ANTAR AI Assistant")

user_input = st.chat_input(
    "How are you feeling today?"
)

if user_input:

    st.chat_message("user").write(
        user_input
    )

    response = (
        "I am here to support you."
    )

    st.chat_message("assistant").write(
        response
    )
```

Purpose:

- Accepts user messages
  - Displays AI conversation
  - Creates real-time chat interaction
- 

## Conclusion

The ANTAR AI Mental Wellness Assistant successfully demonstrates the implementation of a modern AI-powered application using Python technologies.

The project combines secure authentication, AI-generated conversations, database management, responsive user interface, and conversation history management into a single intelligent platform.

The application provides a clean and user-friendly experience through modern ChatGPT-style design and demonstrates practical implementation of frontend, backend, database, and AI integration concepts.